

CS 353-2 Database Systems

Design Report



Group 4

Aral Müftüoğlu
Hüsnü Arkca
Mehmet Dizdar
Onur Ege Yakar
Yunus Emre Aslan

Contents

1.1 Problem Definition.....	3
Overview and Purpose.....	3
Necessity of a Database.....	3
Target Users and Roles.....	3
1.2 System Requirements.....	3
Functional Requirements:.....	3
Non-functional Requirements.....	4
1.3 Limitations.....	4
2.1 Revised E/R Diagram.....	5
2.2 Table Schemas.....	6
2.3 User Interface Design and SQL Queries.....	14
2.3.1 Login and Registration.....	14
2.3.2 Content Browsing & Search.....	17
2.3.3 WatchList Management.....	19
2.3.4 Watch History & Rating.....	22
2.3.5 Social – Follow / Unfollow.....	24
2.3.6 Club Management.....	26
2.3.7 WatchParty.....	29
2.3.8 Group Challenges.....	31
2.3.9 Official Reviews (Critics).....	33
2.3.10 Badge System.....	36
2.3.11 Personalised Feed (Additional Functionality).....	37
2.4 Advanced Database Components.....	42
2.4.1 Views.....	42
2.4.2 Triggers.....	48
2.4.3 Constraints.....	54
2.5 Implementation Plan.....	55

1.1 Problem Definition

Overview and Purpose

The Entertainment Discovery & Analytics Platform is a web-app designed to aggregate movie, TV Show and stream data while creating a social ecosystem for cinematics. Its primary purpose is to move beyond simple "watching" experience by providing advanced discovery tools, verified critical insights, and community-driven "Cinephile Clubs."

Necessity of a Database

For this entertainment platform, a database is necessary because the system is fundamentally built around storing, organizing, and processing large volumes of highly interconnected data. The platform manages movies, TV shows, episodes, users, reviews, ratings, clubs, events, challenges, and follow relationships, all of which are dynamically created and continuously updated by thousands of users. A database provides a structured and reliable way to maintain these complex relationships while ensuring data integrity, consistency, and fast access. Since users perform advanced searches, participate in clubs, and influence scores through ratings (including weighted critic reviews), the system requires powerful querying, aggregation, and transaction capabilities that only a properly designed database management system can provide. Without a database, the platform would suffer from data duplication, inconsistencies, poor performance, and inability to scale.

Target Users and Roles

- **General Viewer:** Standard user; logs history, rates content, joins clubs, and creates lists.
- **Verified Critic:** Critical Experts; provide "Official Reviews" and weighted scores.
- **Club Moderator:** Community leaders; organize events, set challenges, and manage club memberships.
- **Platform Administrator:** System checkers; manage users, handle reports, and generate business analytics.

1.2 System Requirements

Functional Requirements:

- **Watch Logging:** System will record the date and frequency of content viewed by the user. This data can be observed by the user.
- **Event Hosting:** Moderators can schedule "Watch Parties" at specific cinema locations.
- **Follower System:** Users can follow actors/directors to receive notifications of updates on new releases of these people.
- **Custom List Creation:** Users can build and publish public playlists that can be accessible by the other users.
- **Content Search:** Cinematics can be searched using advanced filters such as genre, release year, or director/actor, review score.

- **Weighted Rating System:** System calculates a global score where critics' scores carry more weight than standard users' score.
- **Club Membership System:** Browse or create a cinematic club based on interests (Watched cinematics etc.).
- **Public Profile Page:** Users can view other users' information such as reviews, seen movies if the profile is public.
- **Review System:** Users can write, edit, and delete reviews for movies and TV episodes.
- **Membership Approval:** Moderators can approve or reject club membership requests.
- **Watchlist Feature:** Users can add movies and TV shows to a personal watchlist for future viewing.
- **Short Contents (Extra Feature):** According to Users' watch logs, short contents will be recommended and listed to User's feed. The system will contain these short contents. Short contents will be uploaded by content creators and contain trailers or summaries of original contents.

Non-functional Requirements

- **Scalability:** The database must handle concurrent queries from thousands of users during peak hours.
- **Security:** Role-based access control.
- **Availability:** The platform should be maintained to ensure content is always accessible.
- **Usability:** Query response time for search must be fast to ensure a fluid user experience.

1.3 Limitations

- **No Streaming:** The system tracks and analyzes content but does not host or stream the video files themselves.
- **No Financial Transactions:** While Watch Parties are hosted at cinemas, the system does not handle financial transactions or seat bookings.
- **Watch Log Limitations:** The database tracks what was watched and when, but it will not track "Watch Progress" (for example, paused or resumed at a specific time). Data is stored at the discrete episode or movie level.
- **Overwritten Rating Policy:** A user (General or Critic) can only maintain one active rating per content item. If a user re-rates a movie, the previous rating is overwritten rather than stored as a historical log. This prevents database bloat and simplifies the score calculation.
- **Hierarchical Content:** The system follows Parent-Child dependency. A "TV Episode" cannot exist in the database without being linked to a "TV Series". Orphaned episodes are not supported by the schema.
- **No External Sources:** The system does not use third-party APIs to constantly update contents.

What system will not do:

No Real-time Chat: The social component is restricted to "Cinephile Clubs" and "Lists." There is no instant messaging to avoid the overhead of real-time architecture.

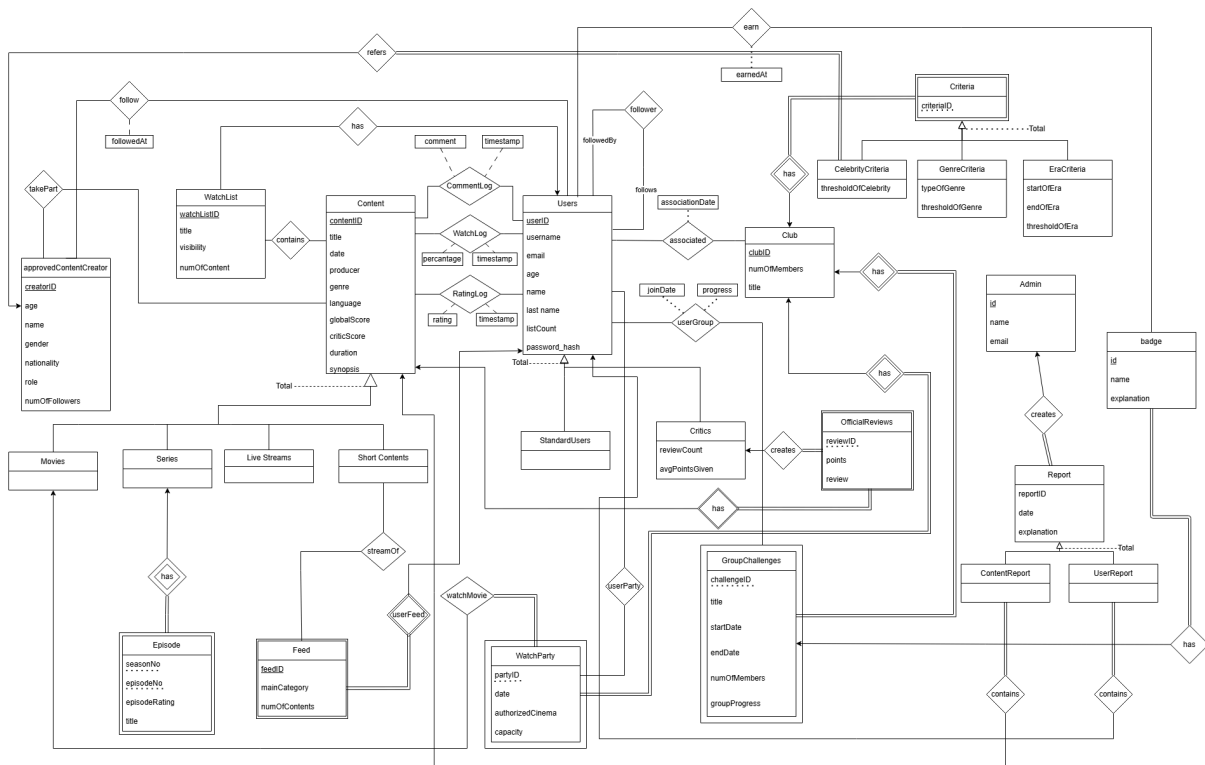
No Real-Time Notification: The system does not provide real-time notifications via SMS, email or any other real-time messaging services.

No Video Playback: The system does not provide an interface for playing videos.

No Advanced Copyright Monitoring: The platform does not implement automated copyright detection mechanisms. Any inappropriate content must be reported and will be handled manually by platform administrators.

No User Ranking System: The platform does not implement a ranking system for general viewers. Users are not evaluated based on follower count, activity level, or review popularity.

2.1 Revised E/R Diagram



Notes

Live Streams mentioned at the E/R diagram are replays of sports matches or documentaries etc. Particular contents that can not be considered as movies or series. (not “live”)

The role approvedContentCreator encapsulates actors and directors. Both have a role at creation of the content and there will be no differences for users when filtering movies by directors or actors. In either way, filtering will be done with the name and role of content creator.

Changes made

- 1) Missing entity -> Admin added
- 2) Missing entity -> Badge added
- 3) Foreign key (moderatorID) removed from Club entity
- 4) User - user follow relationship
- 5) Missing relationship between approvedContentCreator and celebrityCriteria added.
- 6) Missing total participations (watchParty and officialReview) added.
- 7) Users - content relationship is changed to many-to-many
- 8) Password (password_hashed) added to the Users entity
- 9) Log relation splitted into 3 different log types: WatchLog, CommentLog, RatingLog
- 10) Report entity added

For detailed view, the url of the diagram:

https://app.diagrams.net/#G1uH5Dpo6Imq3YBVRRiv5d009_TWSS4bzG%23%7B%22pageId%22%3A%22CowAsM19W_pjbAvAAa6s%22%7D

2.2 Table Schemas

--Users

```
CREATE TABLE Users (  
  userID INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(100) UNIQUE NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  age INT,  
  name VARCHAR(100) NOT NULL,  
  lastName VARCHAR(100) NOT NULL,  
  listCount INT DEFAULT 0,  
  password_hash VARCHAR(255) NOT NULL  
);
```

--StandardUsers

```
CREATE TABLE StandardUsers (  
  userID INT PRIMARY KEY,  
  FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

--Critics

```
CREATE TABLE Critics (  
    userID INT PRIMARY KEY,  
    reviewCount INT DEFAULT 0,  
    avgPointsGiven DECIMAL(5,2),  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

--Follower (User - User self-referential)

```
CREATE TABLE Follower (  
    followerID INT,  
    followedID INT,  
    PRIMARY KEY (followerID, followedID),  
    FOREIGN KEY (followerID) REFERENCES Users(userID) ON DELETE CASCADE,  
    FOREIGN KEY (followedID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

--ApprovedContentCreator

```
CREATE TABLE ApprovedContentCreator (  
    creatorID INT AUTO_INCREMENT PRIMARY KEY,  
    age INT NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    gender VARCHAR(20) NOT NULL,  
    nationality VARCHAR(100) NOT NULL,  
    role VARCHAR(100) NOT NULL,  
    numOfFollowers INT DEFAULT 0  
);
```

--Follow (User - approvedContentCreator)

```
CREATE TABLE Follow(  
    userID INT,  
    creatorID INT,  
    followedAt DATE,  
    PRIMARY KEY (userID, creatorID),  
    FOREIGN KEY (creatorID) REFERENCES ApprovedContentCreator(creatorID) ON DELETE  
CASCADE,  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

--Content

```
CREATE TABLE Content (  
    contentID INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    date DATE,
```

```
    producer VARCHAR(200),  
    genre VARCHAR(100),  
    language VARCHAR(50),  
    globalScore DECIMAL(4,2),  
    criticScore DECIMAL(4,2),  
    duration INT,  
    synopsis TEXT  
);
```

--Movies

```
CREATE TABLE Movies (  
    contentID INT PRIMARY KEY,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--Series

```
CREATE TABLE Series (  
    contentID INT PRIMARY KEY,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--LiveStreams

```
CREATE TABLE LiveStreams (  
    contentID INT PRIMARY KEY,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--ShortContents

```
CREATE TABLE ShortContents (  
    contentID INT PRIMARY KEY,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--Episode

```
CREATE TABLE Episode (  
    contentID INT,  
    seasonNo INT,  
    episodeNo INT,  
    title VARCHAR(200),  
    episodeRating DECIMAL(4,2),  
    PRIMARY KEY (contentID, seasonNo, episodeNo),  
    FOREIGN KEY (contentID) REFERENCES Series(contentID) ON DELETE CASCADE  
);
```

--TakePart (ApprovedContentCreator - Content)

```
CREATE TABLE TakePart (  
    creatorID INT,  
    contentID INT,  
    PRIMARY KEY (creatorID, contentID),  
    FOREIGN KEY (creatorID) REFERENCES ApprovedContentCreator(creatorID) ON DELETE  
CASCADE,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--WatchList

```
CREATE TABLE WatchList (  
    watchListID INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200),  
    visibility VARCHAR(50),  
    numOfContent INT DEFAULT 0,  
    userID INT NOT NULL,  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

--WatchListContent (contains)

```
CREATE TABLE WatchListContent (  
    watchListID INT,  
    contentID INT,  
    PRIMARY KEY (watchListID, contentID),  
    FOREIGN KEY (watchListID) REFERENCES WatchList(watchListID) ON DELETE  
CASCADE,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--CommentLog (User - Content)

```
CREATE TABLE CommentLog (  
    userID INT,  
    contentID INT,  
    timestamp DATETIME,  
    comment TEXT,  
    PRIMARY KEY (userID, contentID),  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE  
);
```

--WatchLog (User - Content)

```
CREATE TABLE WatchLog (  
    userID INT,
```

```

contentID INT,
timestamp DATETIME,
percentage DECIMAL(5,2), --percentage of the amount of watch
PRIMARY KEY (userID, contentID),
FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,
FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE
);

```

--RatingLog (User - Content)

```

CREATE TABLE RatingLog (
    userID INT,
    contentID INT,
    rating INT,
    timestamp DATETIME,
    PRIMARY KEY (userID, contentID),
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE
);

```

--Club

```

CREATE TABLE Club (
    clubID INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    numOfMembers INT DEFAULT 0
);

```

--Admin

```

CREATE TABLE Admin (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(150)
);

```

--Report

```

CREATE TABLE Report(
    reportID INT AUTO_INCREMENT PRIMARY KEY,
    date DATETIME,
    explanation VARCHAR(400) NOT NULL,
    adminID INT,
    FOREIGN KEY (adminID) REFERENCES Admin(id) ON DELETE SET NULL
);

```

--ContentReport

```

CREATE TABLE ContentReport(

```

```

reportID INT PRIMARY KEY,
contentID INT,
FOREIGN KEY (reportID) REFERENCES Report(reportID) ON DELETE CASCADE,
FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE
);

```

--UserReport

```

CREATE TABLE UserReport(
    reportID INT PRIMARY KEY,
    userID INT,
    FOREIGN KEY (reportID) REFERENCES Report(reportID) ON DELETE CASCADE,
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE
);

```

--UserClub (User - Club)

```

CREATE TABLE UserClub (
    userID INT,
    clubID INT,
    joinDate DATE,
    PRIMARY KEY (userID, clubID),
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,
    FOREIGN KEY (clubID) REFERENCES Club(clubID) ON DELETE CASCADE
);

```

--OfficialReviews

```

CREATE TABLE OfficialReviews (
    reviewID INT AUTO_INCREMENT PRIMARY KEY,
    points DECIMAL(5,2),
    review TEXT,
    criticUserID INT NOT NULL,
    contentID INT NOT NULL,
    UNIQUE (criticUserID, contentID),
    FOREIGN KEY (criticUserID) REFERENCES Critics(userID) ON DELETE CASCADE,
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE
);

```

--GroupChallenges

```

CREATE TABLE GroupChallenges (
    challengeID INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100),
    startDate DATE,
    endDate DATE,
    numOfMembers INT DEFAULT 0,
    groupProgress VARCHAR(100),

```

```

clubID INT,
FOREIGN KEY (clubID) REFERENCES Club(clubID) ON DELETE CASCADE
);

```

--UserGroupChallenge (User-GroupChallenge)

```

CREATE TABLE UserGroupChallenge (
    userID INT,
    challengeID INT,
    joinDate DATE,
    progress DECIMAL(5,2), --percentage of progress
    PRIMARY KEY (userID,challengeID),
    FOREIGN KEY (challengeID) REFERENCES GroupChallenges(challengeID) ON DELETE
    CASCADE,
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE
);

```

--Badge

```

CREATE TABLE Badge (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    explanation TEXT,
    challengeID INT,
    FOREIGN KEY (challengeID ) REFERENCES GroupChallenges (challengeID ) ON DELETE
    CASCADE
);

```

--UserBadge (earn)

```

CREATE TABLE UserBadge (
    userID INT,
    badgeID INT,
    earnedAt DATE,
    PRIMARY KEY (userID, badgeID),
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,
    FOREIGN KEY (badgeID) REFERENCES Badge(id) ON DELETE CASCADE
);

```

--Criteria

```

CREATE TABLE Criteria (
    criteriaID INT,
    clubID INT,
    PRIMARY KEY (criteriaID , clubID),
    FOREIGN KEY (clubID) REFERENCES Club(clubID) ON DELETE CASCADE
);

```


--CelebrityCriteria

```
CREATE TABLE CelebrityCriteria (  
    criteriaID INT,  
    clubID INT,  
    thresholdOfCelebrity DECIMAL(10,2),  
    creatorID INT,  
    PRIMARY KEY (criteriaID, clubID),  
    FOREIGN KEY (criteriaID, clubID) REFERENCES Criteria(criteriaID, clubID) ON DELETE  
    CASCADE,  
    FOREIGN KEY (creatorID) REFERENCES ApprovedContentCreator(creatorID) ON DELETE  
    CASCADE  
);
```

--GenreCriteria

```
CREATE TABLE GenreCriteria (  
    criteriaID INT,  
    clubID INT,  
    typeOfGenre VARCHAR(100),  
    thresholdOfGenre DECIMAL(10,2),  
    PRIMARY KEY (criteriaID, clubID),  
    FOREIGN KEY (criteriaID, clubID) REFERENCES Criteria(criteriaID, clubID) ON DELETE  
    CASCADE  
);
```

--EraCriteria

```
CREATE TABLE EraCriteria (  
    criteriaID INT,  
    clubID INT,  
    startOfEra DATE,  
    endOfEra DATE,  
    thresholdOfEra DECIMAL(10,2),  
    PRIMARY KEY (criteriaID, clubID),  
    FOREIGN KEY (criteriaID, clubID) REFERENCES Criteria(criteriaID, clubID) ON DELETE  
    CASCADE  
);
```

--Feed

```
CREATE TABLE Feed (  
    feedID INT AUTO_INCREMENT PRIMARY KEY,  
    mainCategory VARCHAR(100),  
    numOfContents INT DEFAULT 0,  
    userID INT,  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE SET NULL  
);
```

--ShortContent_Feed_Stream (streamOf)

```
CREATE TABLE ShortContent_Feed_Stream (  
    contentID INT,  
    feedID INT,  
    PRIMARY KEY (contentID, feedID),  
    FOREIGN KEY (contentID) REFERENCES ShortContents(contentID) ON DELETE  
CASCADE,  
    FOREIGN KEY (feedID) REFERENCES Feed(feedID) ON DELETE CASCADE  
);
```

--WatchParty

```
CREATE TABLE WatchParty (  
    partyID INT AUTO_INCREMENT PRIMARY KEY,  
    date DATETIME,  
    authorizedCinema VARCHAR(200),  
    capacity INT,  
    contentID INT,  
    clubID INT,  
    FOREIGN KEY (contentID) REFERENCES Content(contentID) ON DELETE CASCADE,  
    FOREIGN KEY (clubID) REFERENCES Club(clubID) ON DELETE CASCADE  
);
```

--WatchPartyUser (userParty)

```
CREATE TABLE WatchPartyUser (  
    partyID INT,  
    userID INT,  
    PRIMARY KEY (partyID, userID),  
    FOREIGN KEY (partyID) REFERENCES WatchParty(partyID) ON DELETE CASCADE,  
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE  
);
```

2.3 User Interface Design and SQL Queries

Each subsection describes a UI screen and the corresponding database operations (SELECT, INSERT, UPDATE, DELETE).

2.3.1 Login and Registration

Login Screen

Fields: Username, Password

Description: The user enters username and password. The system checks whether the credentials match an existing account.

```
SELECT userID, username, email
FROM Users
WHERE username = :username AND password_hash = :password;
```

Note: Passwords are stored as hashed.

Registration Screen

Fields: Username, Email, Password, Name, Last Name, Age, Role (Standard / Critic)

Description: The user enters personal information to create a new account. The system inserts the new user into the Users table and then assigns the selected role by inserting the generated UserID into either StandardUser or Critics.

--Insert new user

```
INSERT INTO Users (username, email, password_hash, age, name, lastName)
VALUES (:username, :email, :password, :age, :name, :lastName);
```

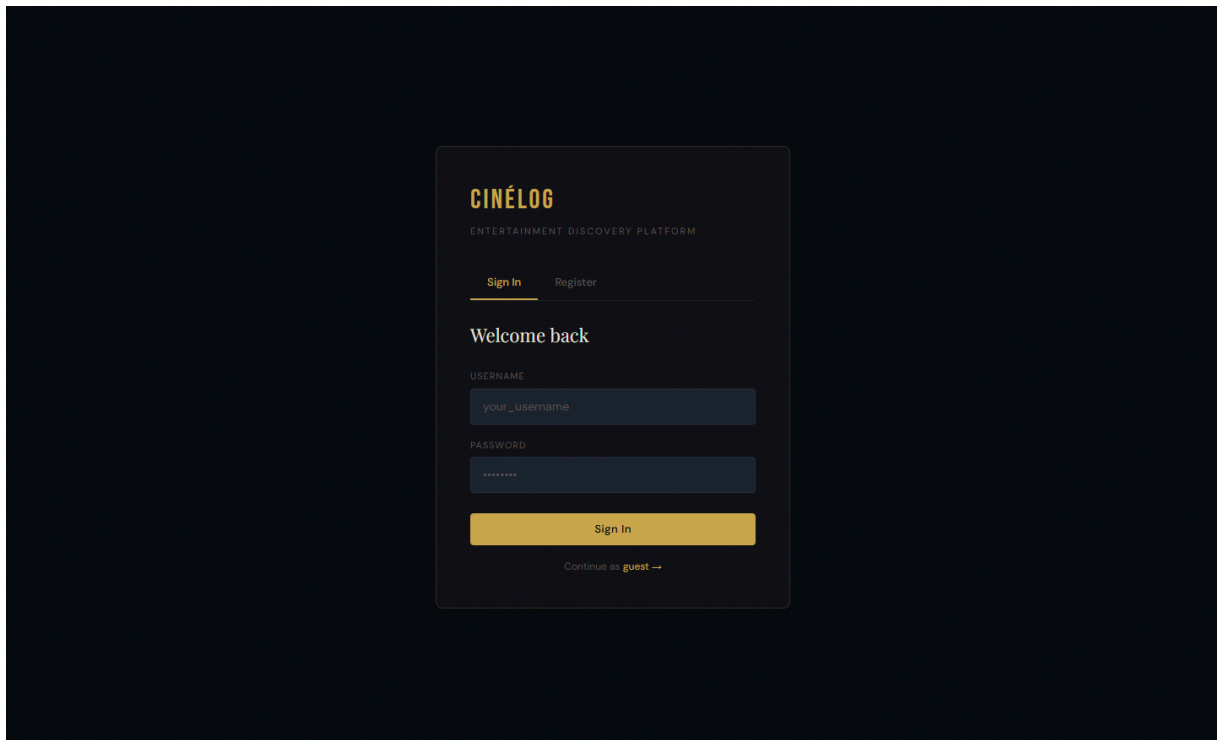
--Assign role for StandardUser

```
INSERT INTO StandardUsers (userID)
VALUES (LAST_INSERT_ID());
```

--OR Assign role for Critic

```
INSERT INTO Critics (userID)
VALUES (LAST_INSERT_ID());
```

Login Interface



The login interface for CINÉLOG, an Entertainment Discovery Platform, is displayed on a dark background. The interface includes a header with the platform name and a subtitle. Below the header, there are two tabs: 'Sign In' (active) and 'Register'. The main heading is 'Welcome back'. The form contains fields for 'USERNAME' (with placeholder 'your_username') and 'PASSWORD' (with placeholder '*****'). A 'Sign In' button is located below the password field. At the bottom, there is a link to 'Continue as guest'.

CINÉLOG
ENTERTAINMENT DISCOVERY PLATFORM

[Sign In](#) [Register](#)

Welcome back

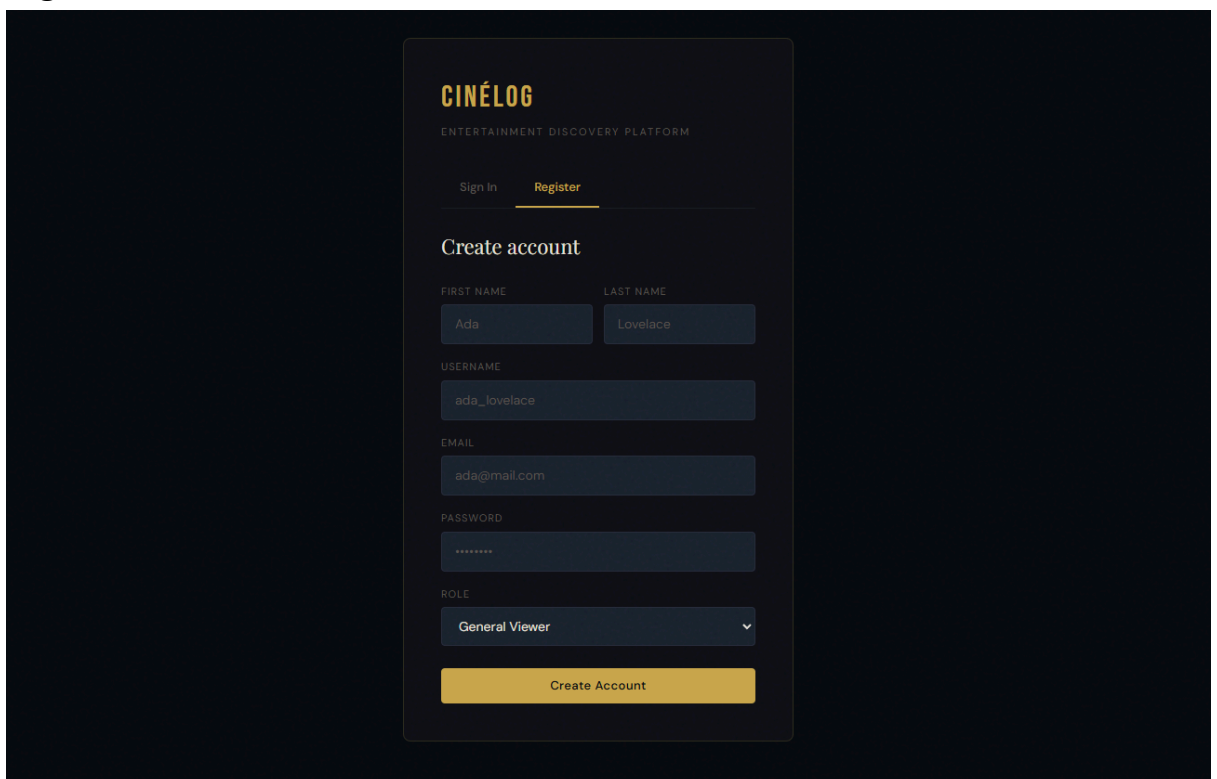
USERNAME
your_username

PASSWORD

Sign In

[Continue as guest](#) →

Register Interface



The register interface for CINÉLOG, an Entertainment Discovery Platform, is displayed on a dark background. The interface includes a header with the platform name and a subtitle. Below the header, there are two tabs: 'Sign In' and 'Register' (active). The main heading is 'Create account'. The form contains fields for 'FIRST NAME' (with placeholder 'Ada'), 'LAST NAME' (with placeholder 'Lovelace'), 'USERNAME' (with placeholder 'ada_lovelace'), 'EMAIL' (with placeholder 'ada@mail.com'), and 'PASSWORD' (with placeholder '*****'). There is also a 'ROLE' dropdown menu with 'General Viewer' selected. A 'Create Account' button is located at the bottom.

CINÉLOG
ENTERTAINMENT DISCOVERY PLATFORM

[Sign In](#) [Register](#)

Create account

FIRST NAME LAST NAME
Ada Lovelace

USERNAME
ada_lovelace

EMAIL
ada@mail.com

PASSWORD

ROLE
General Viewer ▼

Create Account

2.3.2 Content Browsing & Search

Browse / Home Feed

Fields: Genre filter, Language filter, Year filter, Page size, Page number

Description: Displays recently added content sorted primarily by release date and secondarily by global score. The screen supports optional filtering by genre, language, and release year.

```
SELECT c.contentID, c.title, c.genre, c.language,
       c.globalScore, c.criticScore, c.date,
       CASE
         WHEN m.contentID IS NOT NULL THEN 'Movie'
         WHEN s.contentID IS NOT NULL THEN 'Series'
         WHEN l.contentID IS NOT NULL THEN 'LiveStream'
         ELSE 'ShortContent'
       END AS contentType
FROM Content c
LEFT JOIN Movies m ON m.contentID = c.contentID
LEFT JOIN Series s ON s.contentID = c.contentID
LEFT JOIN LiveStreams l ON l.contentID = c.contentID
WHERE (:genre IS NULL OR c.genre = :genre) AND (:language IS NULL OR c.language =
:language) AND (:year IS NULL OR YEAR(c.date) = :year)
ORDER BY c.date DESC, c.globalScore DESC
LIMIT :pageSize OFFSET :offset;
```

Content Detail Page

Fields: contentID

Description: Displays detailed information about a selected content item, including its metadata (title, genre, language, duration, etc.), average user rating, and total number of ratings. The average rating is computed dynamically from user rating logs.

```
SELECT c.contentID, c.title, c.date, c.producer, c.genre, c.language,
       c.globalScore, c.criticScore, c.duration, c.synopsis,
       AVG(rl.rating) AS avgUserRating,
       COUNT(rl.userID) AS totalRatings
FROM Content c
LEFT JOIN RatingLog rl ON rl.contentID = c.contentID
WHERE c.contentID = :contentID
GROUP BY c.contentID, c.title, c.date, c.producer, c.genre, c.language,
         c.globalScore, c.criticScore, c.duration, c.synopsis;
```

View Critic Reviews

Fields: contentID

Description: Displays all official critic reviews for the selected content. Each review includes the critic's username, review text, and rating score. Reviews are sorted in descending order of points.

```
SELECT u.username, cr.points, cr.review
FROM OfficialReviews cr
JOIN Users u ON u.userID = cr.criticUserID
WHERE cr.contentID = :contentID
ORDER BY cr.points DESC;
```

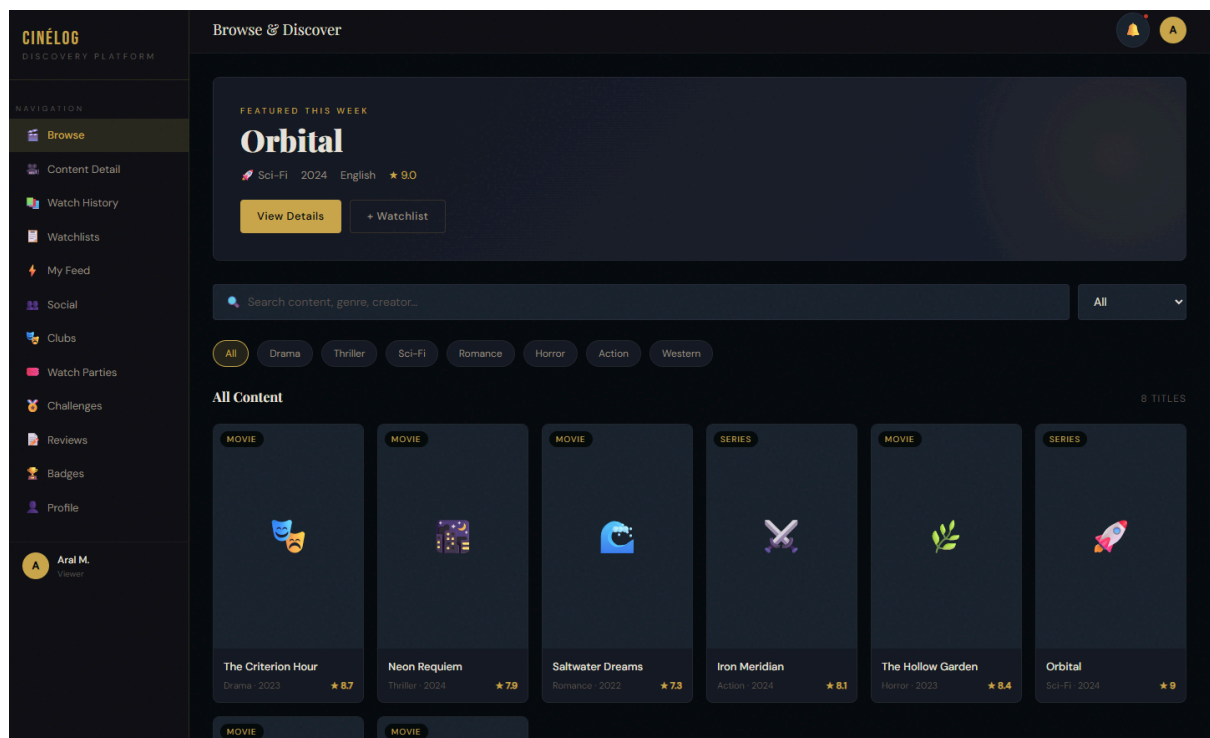
View Cast/Creators

Fields: contentID

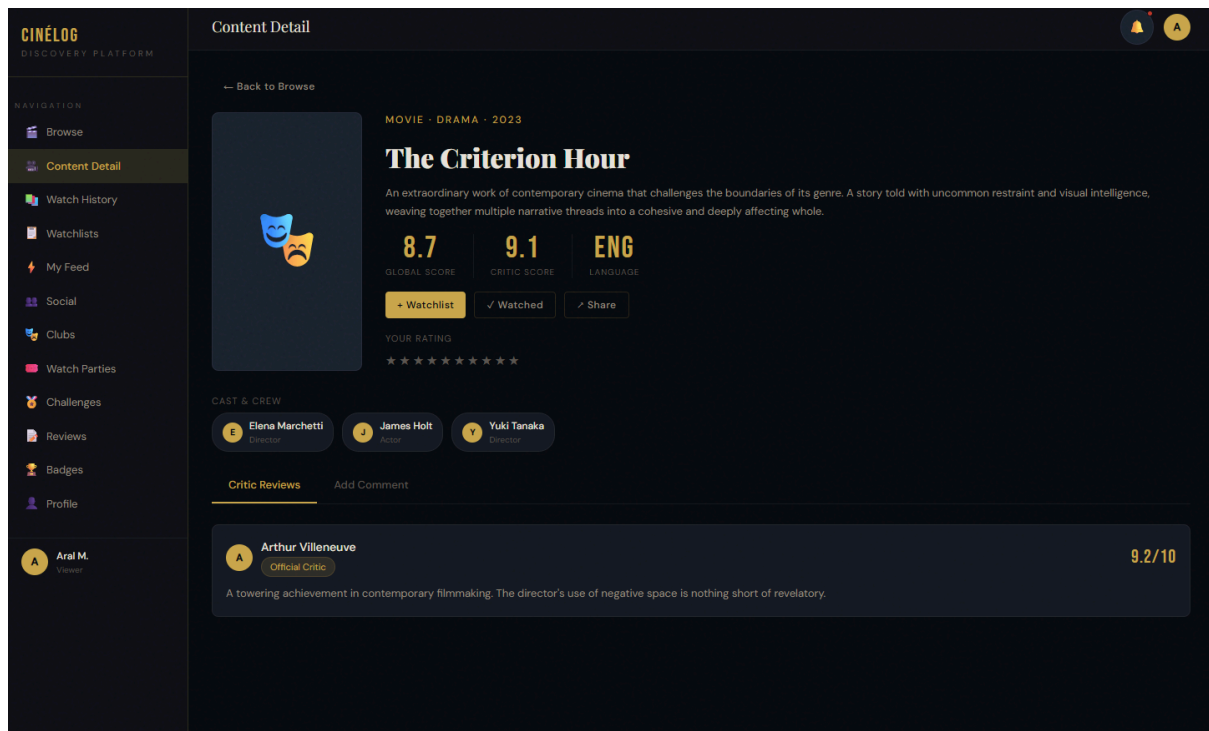
Description: Displays all approved content creators associated with the selected content. Includes information such as creator name and role.

```
SELECT acc.name, acc.role
FROM TakePart tp
JOIN ApprovedContentCreator acc ON acc.creatorID = tp.creatorID
WHERE tp.contentID = :contentID;
```

Home / Browse Interface



View Cast/Creators Interface (+Comments)



2.3.3 WatchList Management

View My WatchLists

Fields: userID

Description: Displays all watchlists created by the user, including watchlist title, visibility, and the number of content items in each list. The results are ordered alphabetically by watchlist title.

```
SELECT wl.watchListID, wl.title, wl.visibility, wl.numOfContent
FROM WatchList wl
WHERE wl.userID = :userID
ORDER BY wl.title;
```

Create WatchList

Fields: userID, title, visibility

Description: Creates a new watchlist for the user with the given title and visibility setting. The watchlist is initialized with zero content items.

```
INSERT INTO WatchList (userID, title, visibility, numOfContent)
VALUES (:userID, :title, :visibility, 0);
```

Add Content to WatchList

Fields: watchListID, contentID

Description: Adds a selected content item to the specified watchlist. After insertion, the number of content items in the watchlist is increased accordingly.

```
INSERT INTO WatchListContent (watchListID, contentID)
VALUES (:watchListID, :contentID);
```

```
UPDATE WatchList
SET numOfContent = numOfContent + 1
WHERE watchListID = :watchListID;
```

Remove Content from WatchList

Fields: watchListID, contentID

Description: Removes a selected content item from the specified watchlist. After deletion, the number of content items in the watchlist is decreased accordingly.

```
DELETE FROM WatchListContent
WHERE watchListID = :watchListID
AND contentID = :contentID;
```

```
UPDATE WatchList
SET numOfContent = numOfContent - 1
WHERE watchListID = :watchListID;
```

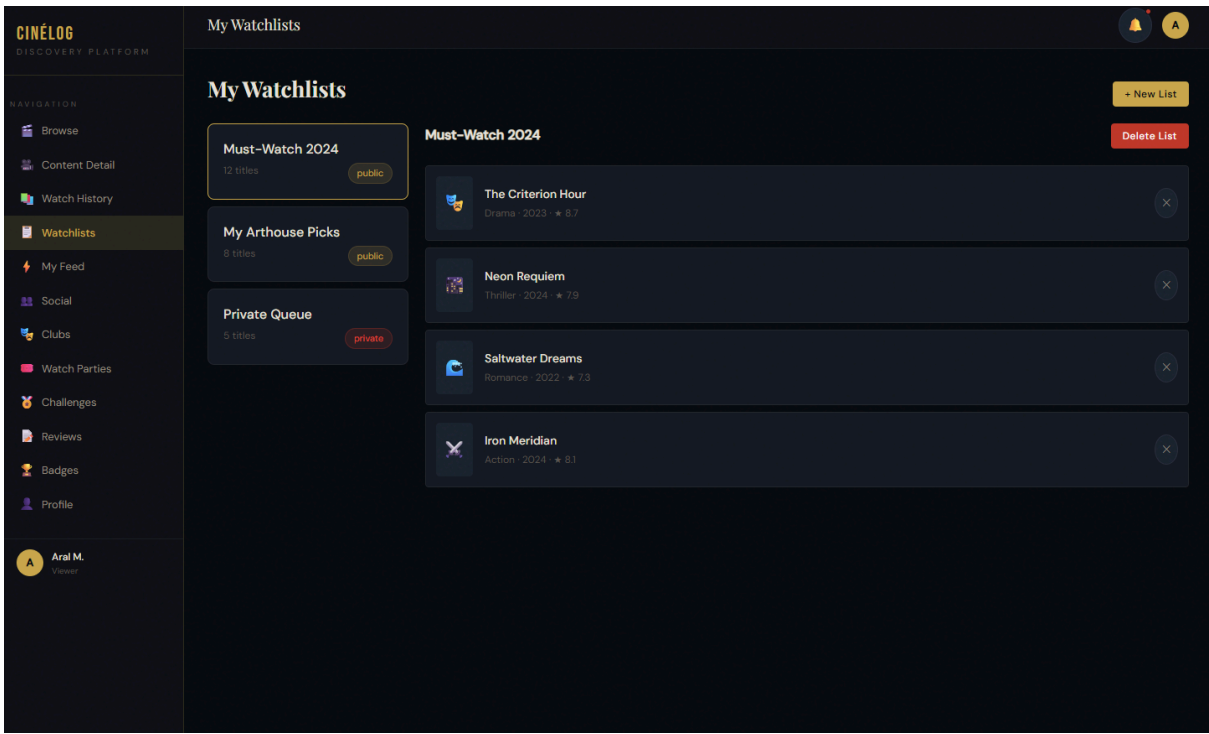
Delete WatchList

Fields: watchListID, userID

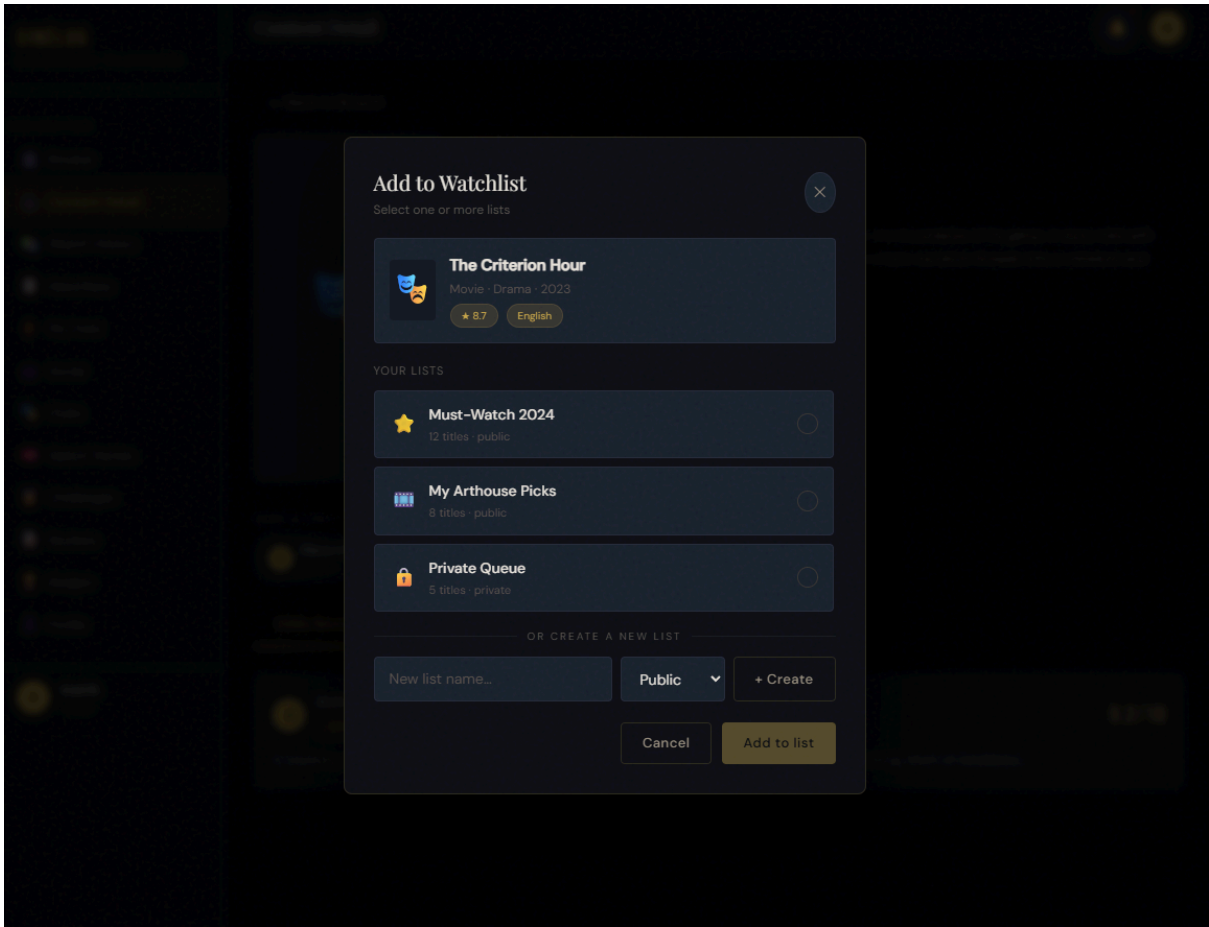
Description: Deletes the specified watchlist belonging to the user. Related watchlist-content associations are also removed automatically through cascading delete constraints.

```
DELETE FROM WatchList
WHERE watchListID = :watchListID
AND userID = :userID;
```


View Watchlists + Movies Inside that Watchlist



Add Movie to Watchlist



2.3.4 Watch History & Rating

Log a Watch / Update Progress

Fields: userID, contentID, percentage

Description: Records or updates the user's watching progress for a content item. Each watch action is logged with the percentage of the content watched.

```
INSERT INTO WatchLog (userID, contentID, timestamp, percentage)
VALUES (:userID, :contentID, NOW(), :percentage);
```

Rate Content

Fields: userID, contentID, rating

Description: Allows the user to rate a content item.

```
INSERT INTO RatingLog (userID, contentID, rating, timestamp)
VALUES (:userID, :contentID, :rating, NOW());
```

Add Comment

Fields: userID, contentID, comment

Description: Allows the user to add a comment for a content item. Each comment is stored with a timestamp.

```
INSERT INTO CommentLog (userID, contentID, timestamp, comment)
VALUES (:userID, :contentID, NOW(), :comment);
```

Update Global Score

Fields: contentID

Description: Updates the global score of a content item based on the average of user ratings.

--Weighted scoring will be implemented in the backend

```
UPDATE Content
SET globalScore = (
    SELECT AVG(r.rating)
    FROM RatingLog r
    WHERE r.contentID = :contentID
)
WHERE contentID = :contentID;
```

Watch History of a User

Fields: userID

Description: Displays the user's watch history, including watched content, progress percentage, rating, and timestamps. The results are ordered by the most recent activity.

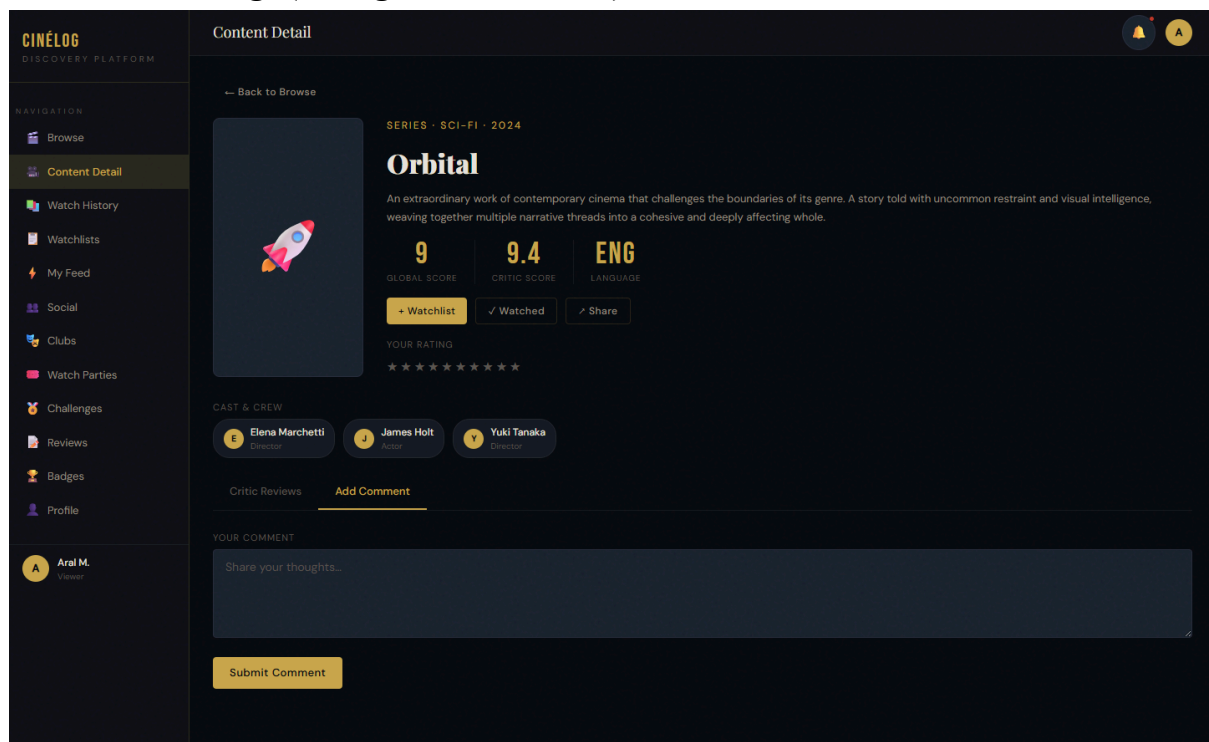
```
SELECT c.title, c.genre,  
       rl.rating,  
       wl.percentage,  
       wl.timestamp AS lastWatched,  
       cl.comment  
FROM WatchLog wl  
JOIN Content c ON c.contentID = wl.contentID  
LEFT JOIN RatingLog rl  
  ON rl.userID = wl.userID AND rl.contentID = wl.contentID  
LEFT JOIN CommentLog cl  
  ON cl.userID = wl.userID AND cl.contentID = wl.contentID  
WHERE wl.userID = :userID  
ORDER BY wl.timestamp DESC;
```

Watch History Page

The screenshot shows the 'Watch History' page on the CINÉLOG platform. The left sidebar contains navigation links: Browse, Content Detail, Watch History (selected), Watchlists, My Feed, Social, Clubs, Watch Parties, Challenges, Reviews, Badges, and Profile. The user's name 'Aral M.' is visible at the bottom of the sidebar. The main content area is titled 'Watch History' and shows a summary of 5 titles with the following statistics: 47 Total Watched, 8.1 Avg Rating, Drama as the Top Genre, and 312h Watch Time. Below the summary, a list of watched titles is displayed with their progress bars, ratings, and genres:

Title	Genre	Date	Progress	Rating	Notes
Orbital	Sci-Fi	2024-07-28	100%	★ 9/10	"Breathtaking visuals"
Neon Requiem	Thriller	2024-07-22	100%	★ 8/10	
The Criterion Hour	Drama	2024-07-18	67%	Not rated	
Marble & Dust	Drama	2024-07-10	100%	★ 8/10	"Slow but rewarding"
Saltwater Dreams	Romance	2024-07-01	100%	★ 7/10	

Content Detail Page (Rating is included here)



2.3.5 Social – Follow / Unfollow

Follow a User

Fields: followerID, followedID

Description: Allows a user to follow another user. Duplicate follow relationships are ignored if the relationship already exists.

INSERT INTO Follower (followerID, followedID)

VALUES (:followerID, :followedID)

ON DUPLICATE KEY UPDATE followerID = followerID;

Unfollow a User

Fields: followerID, followedID

Description: Removes an existing follow relationship between two users.

DELETE FROM Follower

WHERE followerID = :followerID
AND followedID = :followedID;

Get Followers List

Fields: userID

Description: Displays the list of users who follow the specified user.

```
SELECT u.userID, u.username  
FROM Follower f  
JOIN Users u ON u.userID = f.followerID  
WHERE f.followedID = :userID;
```

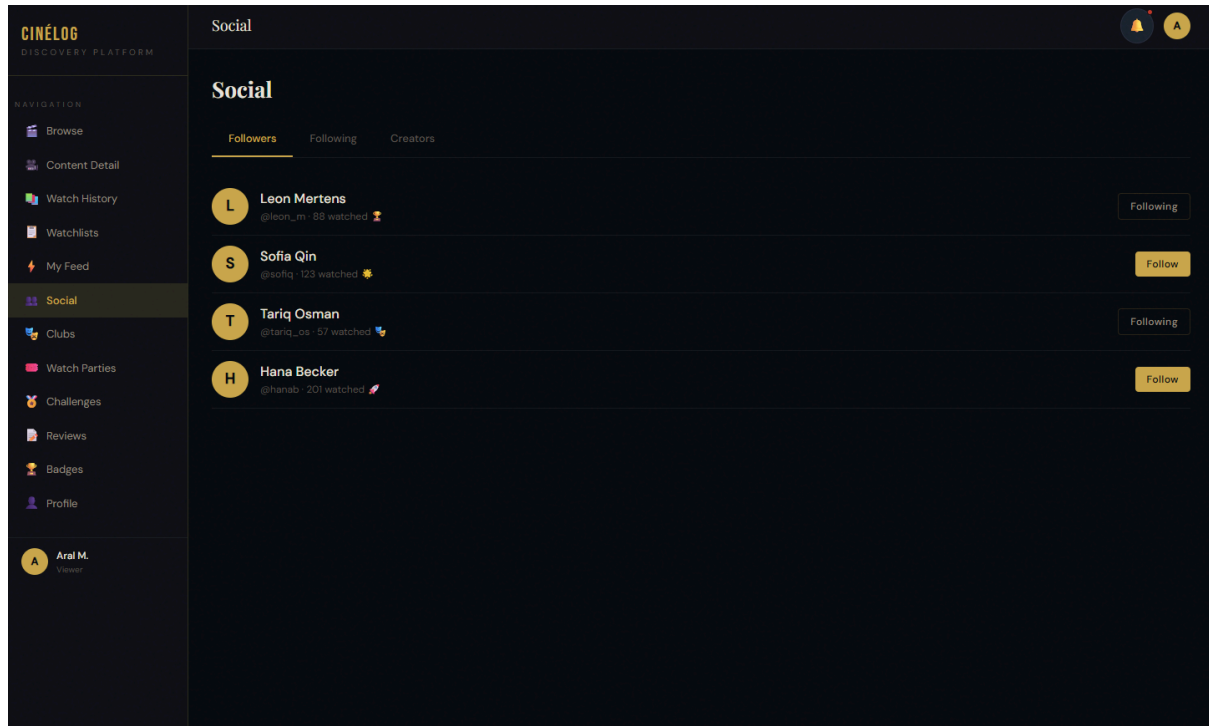
Get Following List

Fields: userID

Description: Displays the list of users that the specified user follows.

```
SELECT u.userID, u.username  
FROM Follower f  
JOIN Users u ON u.userID = f.followedID  
WHERE f.followerID = :userID;
```

Follow - Unfollow - Followers Interface



2.3.6 Club Management

Browse Clubs

Fields: –

Description: Displays all clubs along with their titles and number of members. Clubs are ordered by the number of members in descending order.

```
SELECT c.clubID, c.title, c.numOfMembers
FROM Club c
ORDER BY c.numOfMembers DESC;
```

Join a Club

Fields: userID, clubID

Description: Allows a user to join a club. The membership is recorded with the join date. The number of members in the club is increased accordingly.

```
INSERT INTO UserClub (userID, clubID, joinDate)
VALUES (:userID, :clubID, CURRENT_DATE);
```

```
UPDATE Club
SET numOfMembers = numOfMembers + 1
```

WHERE clubID = :clubID;

Leave a Club

Fields: userID, clubID

Description: Allows a user to leave a club. The membership record is removed and the number of members is decreased accordingly.

```
DELETE FROM UserClub
WHERE userID = :userID
AND clubID = :clubID;
```

```
UPDATE Club
SET numOfMembers = GREATEST(numOfMembers - 1, 0)
WHERE clubID = :clubID;
```

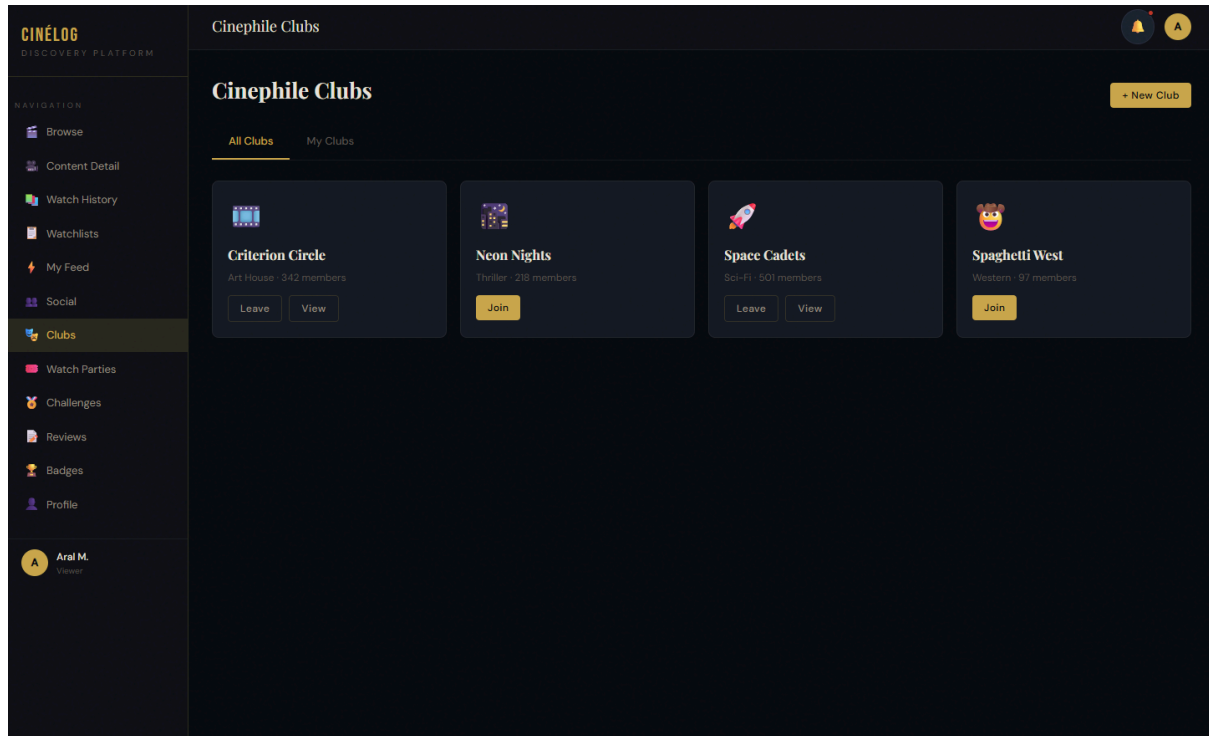
Club Member List

Fields: clubID

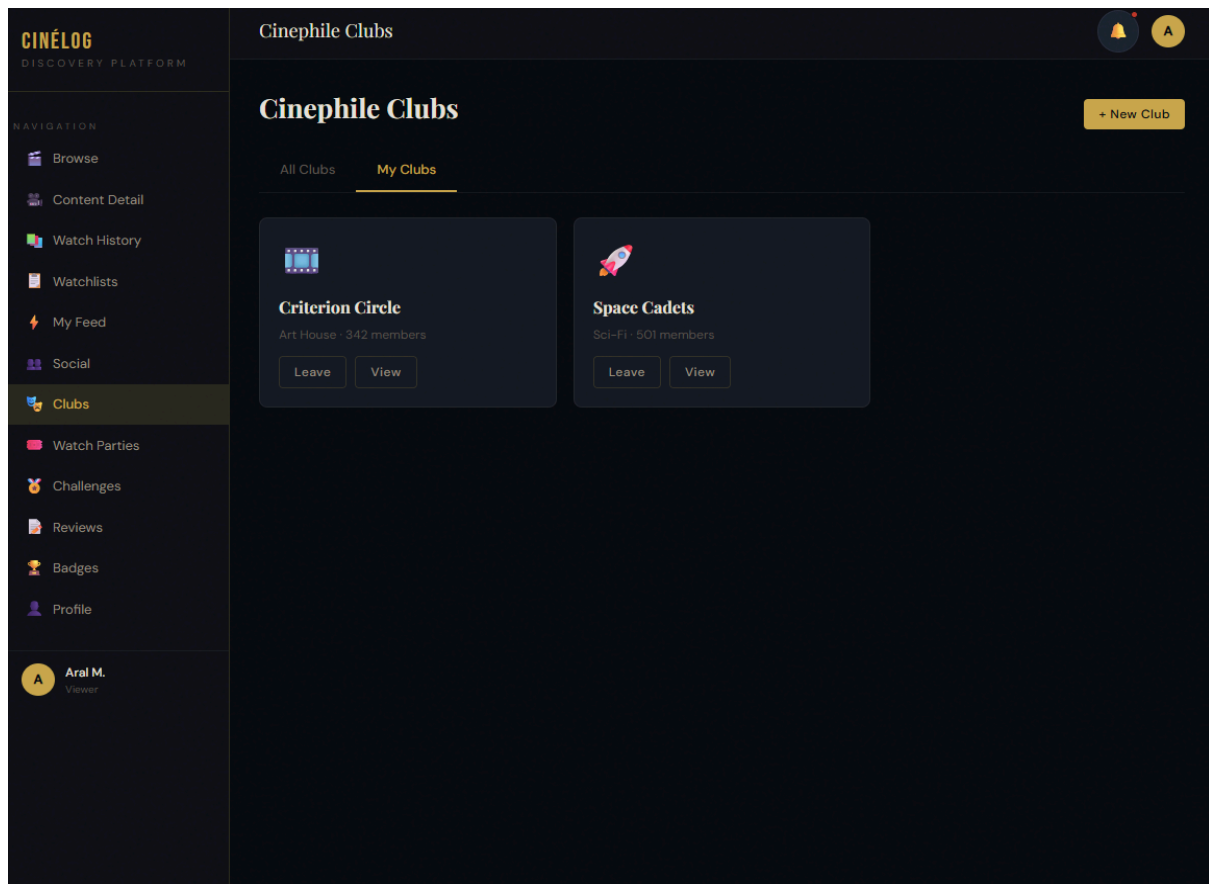
Description: Displays the list of members in a club along with their usernames and join dates. The results are ordered by join date.

```
SELECT u.username, uc.joinDate
FROM UserClub uc
JOIN Users u ON u.userID = uc.userID
WHERE uc.clubID = :clubID
ORDER BY uc.joinDate;
```

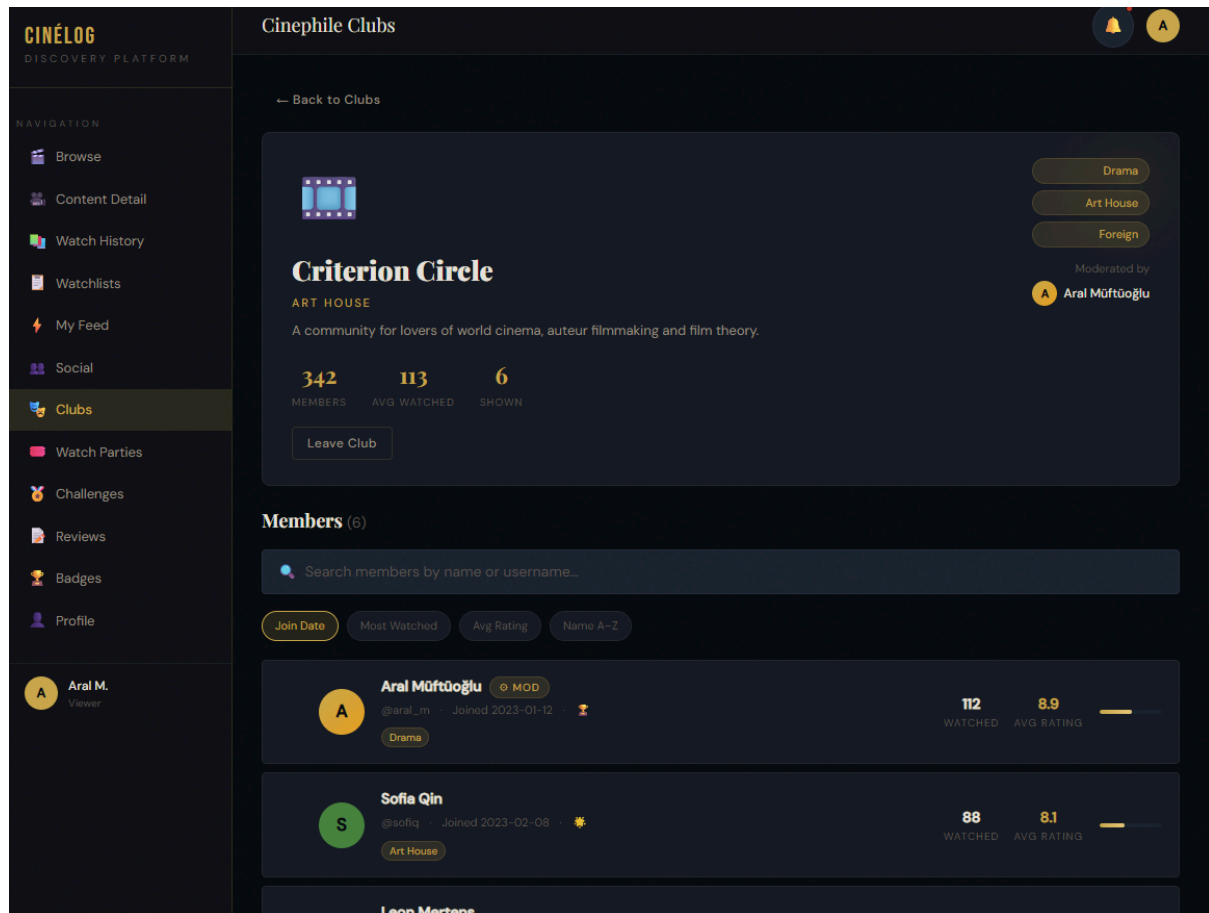
Club Page (All clubs - browse)



My Clubs



Club Member List Interface



2.3.7 WatchParty

Create WatchParty

Fields: contentID, clubID, date, authorizedCinema, capacity

Description: Creates a new watch party for a selected content item within a club. The event stores the date, authorized cinema, and maximum capacity.

```
INSERT INTO WatchParty (contentID, clubID, date, authorizedCinema, capacity)
VALUES (:contentID, :clubID, :date, :cinema, :capacity);
```

Join WatchParty

Fields: partyID, userID

Description: Allows a user to join a watch party if the party has not reached its capacity. The current number of joined users is checked before insertion.

-- Check capacity before joining

```
SELECT wp.capacity,
```

```

COUNT(wpu.userID) AS currentParticipants
FROM WatchParty wp
LEFT JOIN WatchPartyUser wpu ON wpu.partyID = wp.partyID
WHERE wp.partyID = :partyID
GROUP BY wp.partyID, wp.capacity;

```

```

INSERT INTO WatchPartyUser (partyID, userID)
VALUES (:partyID, :userID);

```

List Upcoming WatchParties

Fields: -

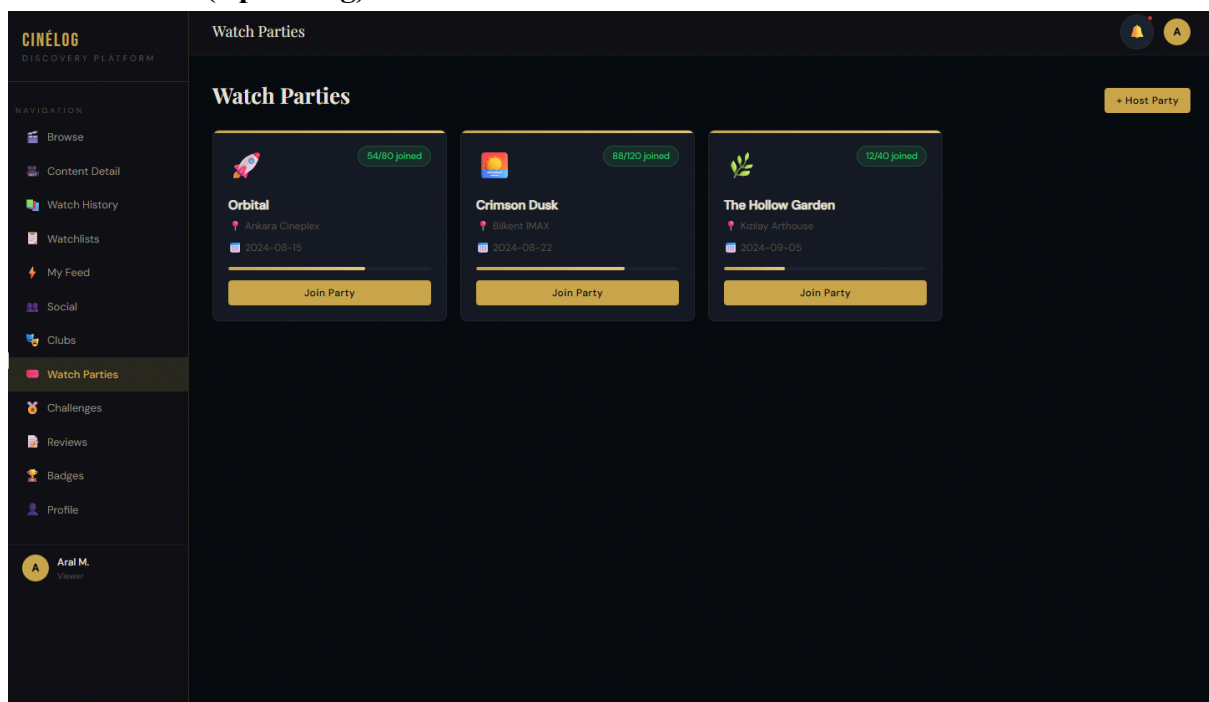
Description: Displays all upcoming watch parties together with their associated content titles, dates, cinema information, capacities, and current participant counts.

```

SELECT wp.partyID, c.title, wp.date, wp.authorizedCinema,
       wp.capacity,
       COUNT(wpu.userID) AS joined
FROM WatchParty wp
JOIN Content c ON c.contentID = wp.contentID
LEFT JOIN WatchPartyUser wpu ON wpu.partyID = wp.partyID
WHERE wp.date > NOW()
GROUP BY wp.partyID, c.title, wp.date, wp.authorizedCinema, wp.capacity
ORDER BY wp.date;

```

Watch Parties (Upcoming)



2.3.8 Group Challenges

Create Group Challenge

Fields: clubID, title, startDate, endDate

Description: Creates a new group challenge for a club. The challenge is initialized with zero members and a default progress status.

```
INSERT INTO GroupChallenges (clubID, title, startDate, endDate, numOfMembers,
groupProgress)
VALUES (:clubID, :title, :startDate, :endDate, 0, 'Not Started');
```

View Active Challenges for a Club

Fields: clubID

Description: Displays all active group challenges of a club whose end date has not yet passed.

```
SELECT gc.challengeID, gc.title, gc.startDate, gc.endDate,
       gc.numOfMembers, gc.groupProgress
FROM GroupChallenges gc
WHERE gc.clubID = :clubID
      AND gc.endDate >= CURRENT_DATE
ORDER BY gc.startDate;
```

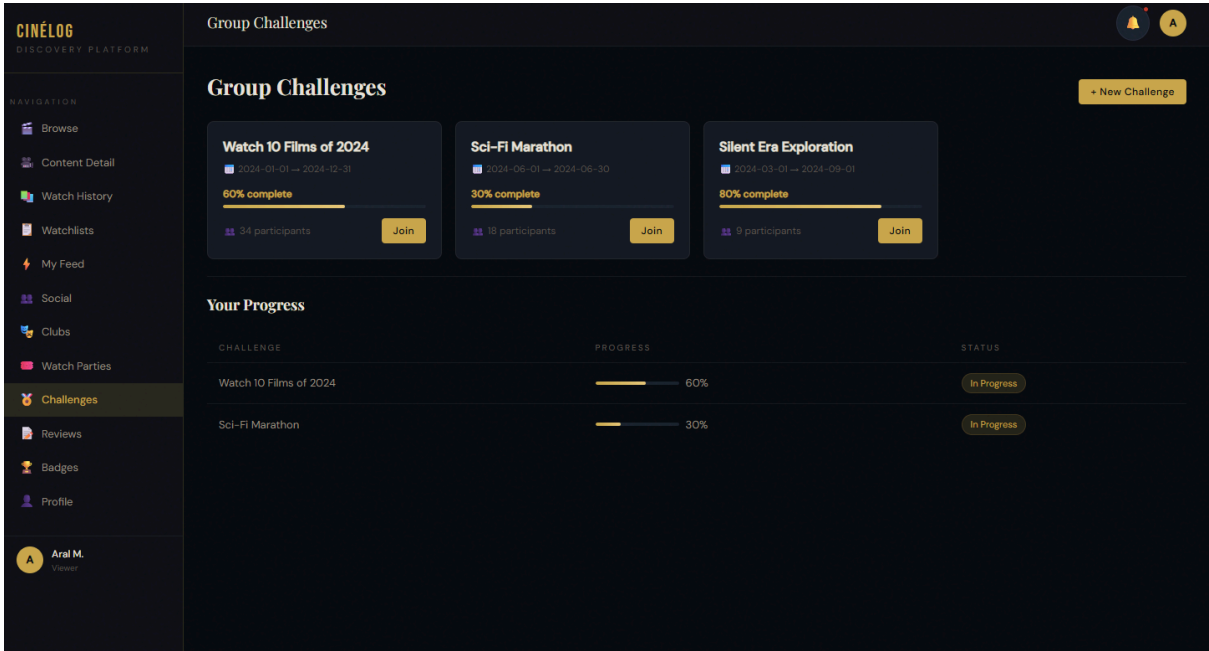
Update Challenge Progress

Fields: challengeID, progress

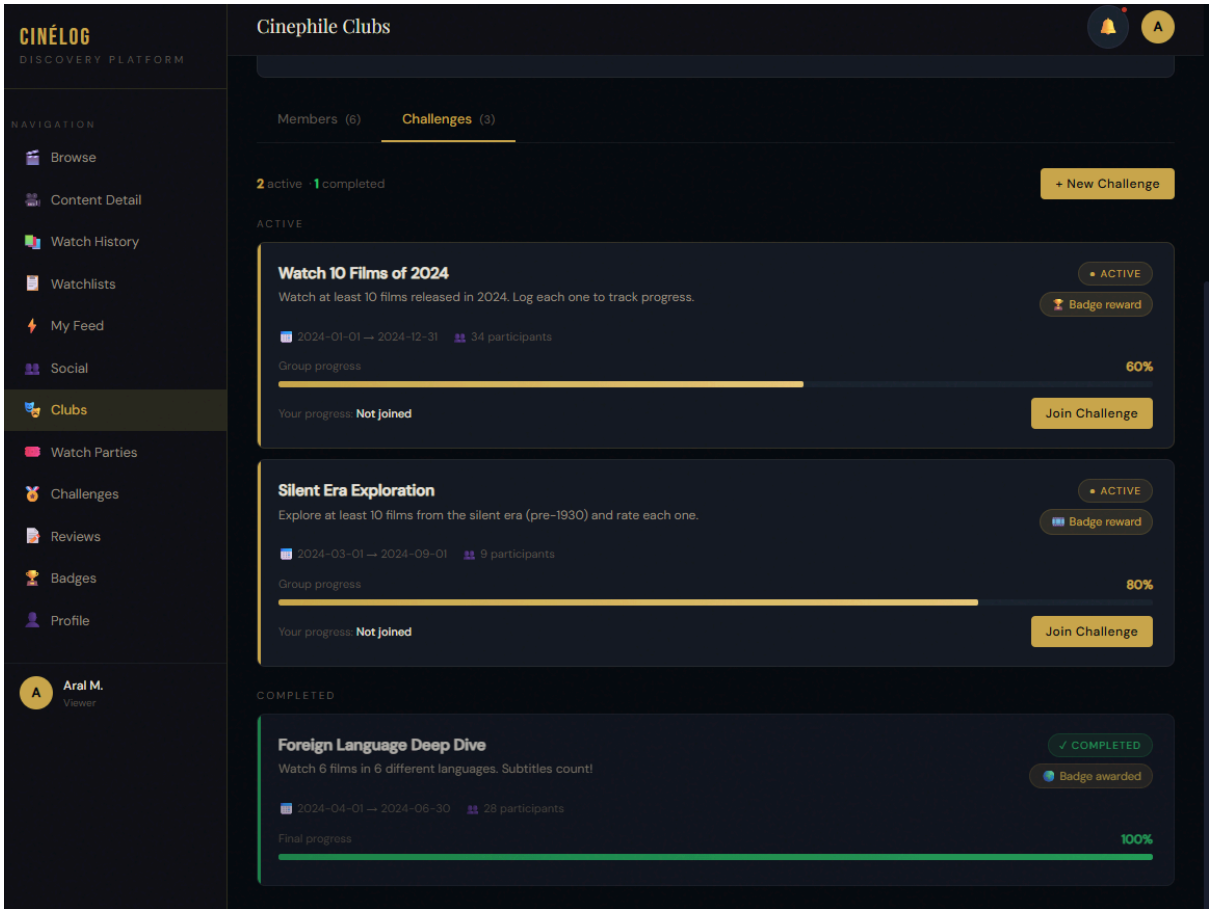
Description: Updates the progress status of a group challenge.

```
UPDATE GroupChallenges
SET groupProgress = :progress
WHERE challengeID = :challengeID;
```

Group Challenges



Club Group Challenges Interface



Create New Group Challenge Interface

New Group Challenge

CHALLENGE TITLE
e.g. Summer Horror Marathon

DESCRIPTION
What do members need to do?

START DATE
gg/aa/yyyy

END DATE
gg/aa/yyyy

BADGE REWARD (EMOJI)
e.g. 🏆

Cancel Create Challenge

2.3.9 Official Reviews (Critics)

Submit Review

Fields: criticUserID, contentID, points, reviewText

Description: Allows a critic to submit an official review for a content item. After insertion, critic statistics and the content's critic score are refreshed.

--If there are no existing review by the critic for the content

```
INSERT INTO OfficialReviews (criticUserID, contentID, points, review)
VALUES (:criticUserID, :contentID, :points, :reviewText);
```

--If there is an existing review by the critic for the content, it will be updated

```
UPDATE OfficialReviews
SET points = :points,
reviewText = :reviewText
```

```
WHERE criticUserID = :criticUserID AND contentID = :contentID;
```

-- Update critic statistics

```
UPDATE Critics
SET reviewCount = reviewCount + 1,
    avgPointsGiven = (
        SELECT AVG(points)
        FROM OfficialReviews
        WHERE criticUserID = :criticUserID
    )
WHERE userID = :criticUserID;
```

-- Refresh criticScore on Content

```
UPDATE Content
SET criticScore = (
    SELECT AVG(points)
    FROM OfficialReviews
    WHERE contentID = :contentID
)
WHERE contentID = :contentID;
```

Edit Review

Fields: reviewID, criticUserID, points, reviewText

Description: Updates the points and review text of an existing official review.

```
UPDATE OfficialReviews
SET points = :points,
    review = :reviewText
WHERE reviewID = :reviewID
AND criticUserID = :criticUserID;
```

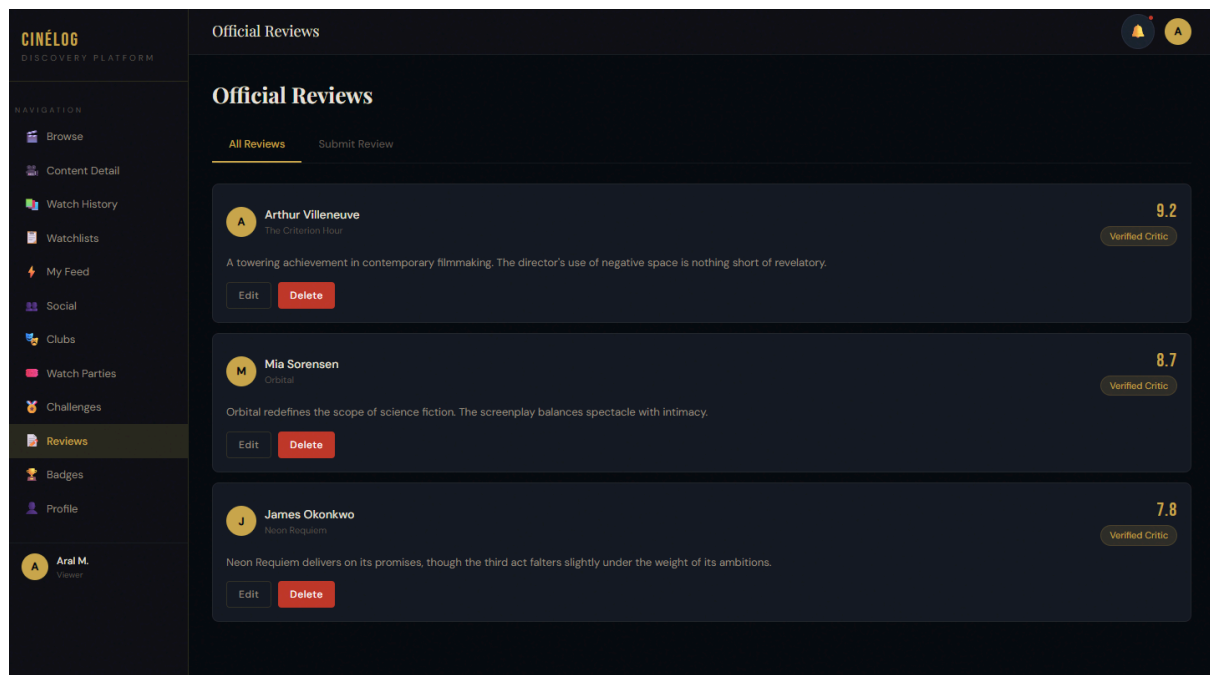
Delete Review

Fields: reviewID, criticUserID

Description: Deletes an official review submitted by a critic.

```
DELETE FROM OfficialReviews
WHERE reviewID = :reviewID
AND criticUserID = :criticUserID;
```

All Official Reviews



Create + Edit Review Interface

2.3.10 Badge System

View Earned Badges

Fields: userID

Description: Displays all badges earned by a user together with badge names and explanations.

```
SELECT b.name, b.explanation
FROM UserBadge ub
JOIN Badge b ON b.id = ub.badgeID
WHERE ub.userID = :userID;
```

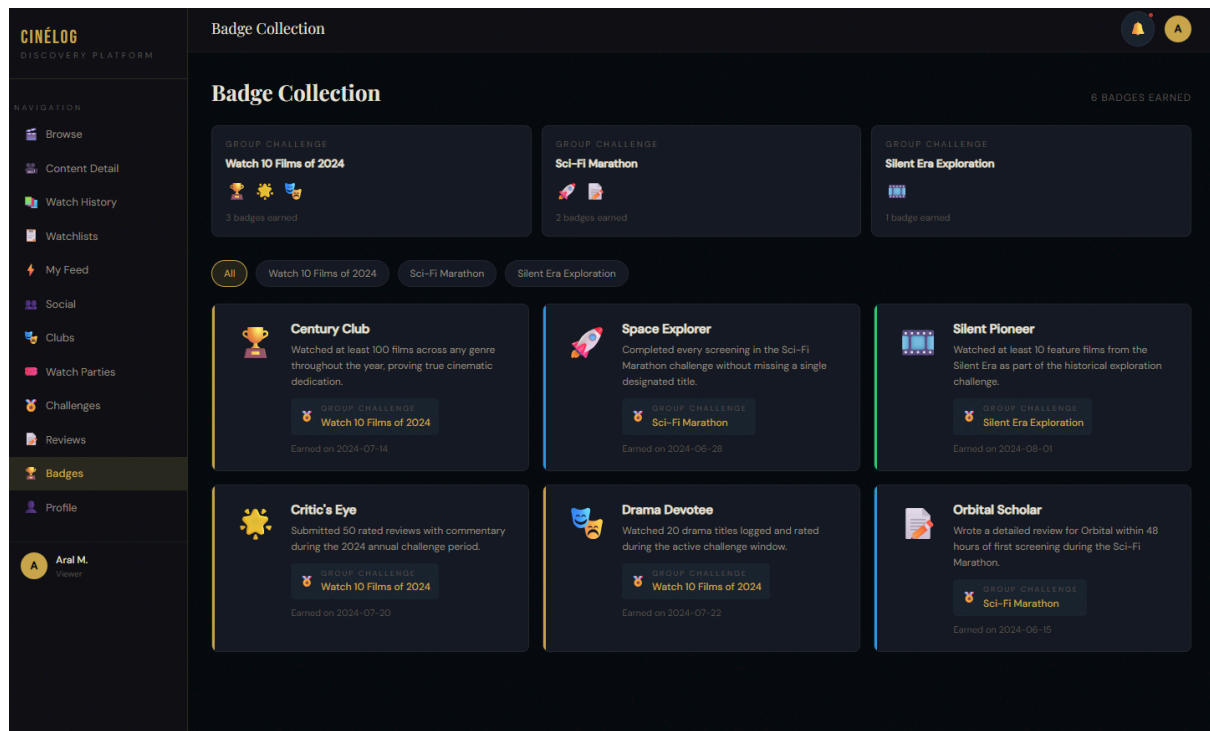
Award Badge to User

Fields: userID, badgeID

Description: System assigns a badge to a user if the badge has not already been awarded.

```
INSERT IGNORE INTO UserBadge (userID, badgeID, earnedAt)
VALUES (:userID, :badgeID, CURRENT_DATE);
```


User's Badge Collection



2.3.11 Personalised Feed (Additional Functionality)

View Feed Contents

Fields: userID

Description: Displays the contents of a user's personalised feed together with content information and the feed's main category. Results are ordered by global score.

```
SELECT c.contentID, c.title, c.genre, c.globalScore, f.mainCategory
FROM Feed f
JOIN ShortContent_Feed_Stream sfs ON sfs.feedID = f.feedID
JOIN Content c ON c.contentID = sfs.contentID
WHERE f.userID = :userID
ORDER BY c.globalScore DESC;
```

Update Feed Category

Get User's Favourite Genre and Content Creators (Actors or Directors) to Form a Feed

```

SELECT c.genre, COUNT(*) AS watchCount
FROM WatchLog w
JOIN Content c ON c.contentID = w.contentID
WHERE w.userID = :userID AND c.genre IS NOT NULL
GROUP BY c.genre
ORDER BY watchCount DESC
LIMIT 3;

```

```

SELECT acc.creatorID, acc.name, acc.role, COUNT(*) AS watchCount
FROM WatchLog w
JOIN TakePart tp ON tp.contentID = w.contentID
JOIN ApprovedContentCreator acc ON acc.creatorID = tp.creatorID
WHERE w.userID = :userID
GROUP BY acc.creatorID, acc.name, acc.role
ORDER BY watchCount DESC
LIMIT 3;

```

Get Target Short Contents for the Feed and Insert to Feed

Description: Get 100 short contents that are parallel to user's preferences, which are obtained from user's watch logs. The distribution will be 30 - 15 - 5 for both genre and content creators. So the feed will be formed with 100 contents, 50% will be determined by top 3 genres and 50% will be determined by top 3 content creators (actor or director).

Create User's Feed

```

INSERT INTO Feed (mainCategory, numOfContents, userID)
VALUES ('Personalised', 100, :userID);

```

Insert Short Contents to Feed with Decided Distribution

```

INSERT INTO ShortContent_Feed_Stream (contentID, feedID)
SELECT contentID, (SELECT MAX(feedID) FROM Feed WHERE userID = :userID)
FROM (

    (SELECT c.contentID
     FROM ShortContents sc
     JOIN Content c ON c.contentID = sc.contentID
     WHERE c.genre = (
         SELECT c2.genre
         FROM WatchLog wl JOIN Content c2 ON c2.contentID = wl.contentID
         WHERE wl.userID = :userID AND c2.genre IS NOT NULL
         GROUP BY c2.genre ORDER BY COUNT(*) DESC LIMIT 1
     )
    )
)

```

ORDER BY c.globalScore DESC LIMIT 30)

UNION ALL

```
(SELECT c.contentID
FROM ShortContents sc
JOIN Content c ON c.contentID = sc.contentID
JOIN TakePart tp ON tp.contentID = sc.contentID
WHERE tp.creatorID = (
    SELECT acc.creatorID
    FROM WatchLog wl
    JOIN TakePart tp2 ON tp2.contentID = wl.contentID
    JOIN ApprovedContentCreator acc ON acc.creatorID = tp2.creatorID
    WHERE wl.userID = :userID
    GROUP BY acc.creatorID ORDER BY COUNT(*) DESC LIMIT 1
)
ORDER BY c.globalScore DESC LIMIT 30)
```

UNION ALL

```
(SELECT c.contentID
FROM ShortContents sc
JOIN Content c ON c.contentID = sc.contentID
WHERE c.genre = (
    SELECT genre FROM (
        SELECT c2.genre, COUNT(*) AS cnt
        FROM WatchLog wl JOIN Content c2 ON c2.contentID = wl.contentID
        WHERE wl.userID = :userID AND c2.genre IS NOT NULL
        GROUP BY c2.genre ORDER BY cnt DESC LIMIT 2
    ) AS top2Genres
    ORDER BY cnt ASC LIMIT 1
)
ORDER BY c.globalScore DESC LIMIT 15)
```

UNION ALL

```
(SELECT c.contentID
FROM ShortContents sc
JOIN Content c ON c.contentID = sc.contentID
JOIN TakePart tp ON tp.contentID = sc.contentID
WHERE tp.creatorID = (
    SELECT creatorID FROM (
        SELECT acc.creatorID, COUNT(*) AS cnt
        FROM WatchLog wl
```

```

        JOIN TakePart tp2 ON tp2.contentID = wl.contentID
        JOIN ApprovedContentCreator acc ON acc.creatorID = tp2.creatorID
        WHERE wl.userID = :userID
        GROUP BY acc.creatorID ORDER BY cnt DESC LIMIT 2
    ) AS top2Creators
    ORDER BY cnt ASC LIMIT 1
)
ORDER BY c.globalScore DESC LIMIT 15)

UNION ALL

(SELECT c.contentID
FROM ShortContents sc
JOIN Content c ON c.contentID = sc.contentID
WHERE c.genre = (
    SELECT genre FROM (
        SELECT c2.genre, COUNT(*) AS cnt
        FROM WatchLog wl JOIN Content c2 ON c2.contentID = wl.contentID
        WHERE wl.userID = :userID AND c2.genre IS NOT NULL
        GROUP BY c2.genre ORDER BY cnt DESC LIMIT 3
    ) AS top3Genres
    ORDER BY cnt ASC LIMIT 1
)
ORDER BY c.globalScore DESC LIMIT 5)

UNION ALL

(SELECT c.contentID
FROM ShortContents sc
JOIN Content c ON c.contentID = sc.contentID
JOIN TakePart tp ON tp.contentID = sc.contentID
WHERE tp.creatorID = (
    SELECT creatorID FROM (
        SELECT acc.creatorID, COUNT(*) AS cnt
        FROM WatchLog wl
        JOIN TakePart tp2 ON tp2.contentID = wl.contentID
        JOIN ApprovedContentCreator acc ON acc.creatorID = tp2.creatorID
        WHERE wl.userID = :userID
        GROUP BY acc.creatorID ORDER BY cnt DESC LIMIT 3
    ) AS top3Creators
    ORDER BY cnt ASC LIMIT 1
)
ORDER BY c.globalScore DESC LIMIT 5)

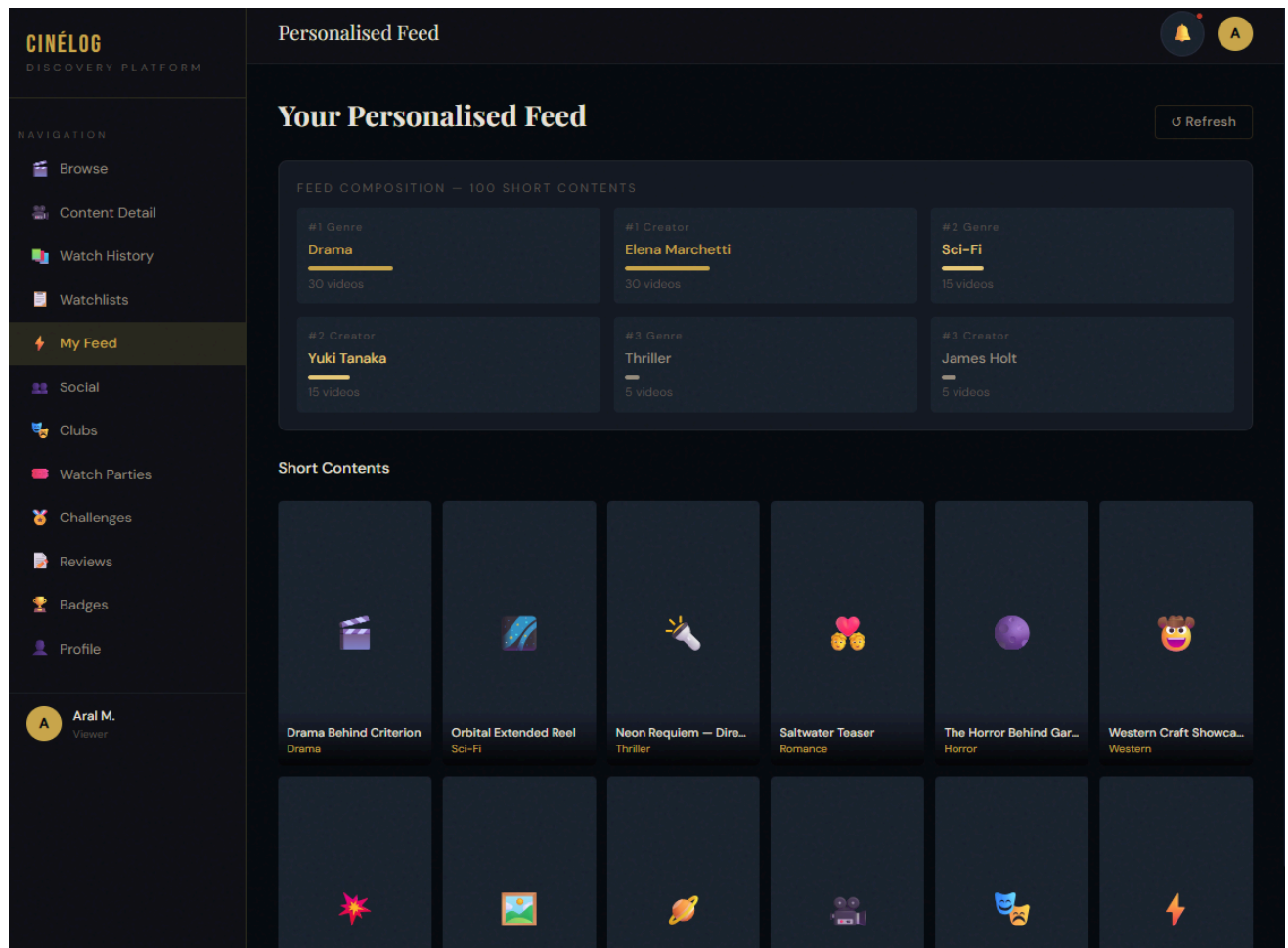
```

```
) AS feed  
ORDER BY RAND();
```

Update Number of Contents to Eliminate Mismatches

```
UPDATE Feed  
SET numOfContents = (  
    SELECT COUNT(*)  
    FROM ShortContent_Feed_Stream  
    WHERE feedID = (SELECT MAX(feedID) FROM Feed WHERE userID = :userID)  
)  
WHERE feedID = (SELECT MAX(feedID) FROM Feed WHERE userID = :userID);
```

User Feed Interface



2.4 Advanced Database Components

2.4.1 Views

View 1 – ContentStatistics:

Purpose

Provides a unified statistics summary for each content item, combining user ratings from Log and critic ratings from OfficialReviews. Avoids repeating complex aggregation joins in every application query and keeps scoring logic centralised.

Use Case

Used in content detail pages, recommendation systems, and ranking queries (e.g., top-rated movies).

View:

```
CREATE VIEW ContentStatistics AS
SELECT
    c.contentID,
    c.title,
    c.genre,
    c.language,
    c.duration,
    c.date,
    ROUND(AVG(rl.rating), 2) AS avgUserRating,
    COUNT(rl.userID) AS totalUserRatings,
    ROUND(AVG(or2.points), 2) AS avgCriticScore,
    COUNT(or2.reviewID) AS totalCriticReviews,
    CASE
        WHEN m.contentID IS NOT NULL THEN 'Movie'
        WHEN s.contentID IS NOT NULL THEN 'Series'
        WHEN ls.contentID IS NOT NULL THEN 'LiveStream'
        ELSE 'ShortContent'
    END AS contentType
FROM Content c
LEFT JOIN RatingLog rl ON rl.contentID = c.contentID
LEFT JOIN OfficialReviews or2 ON or2.contentID = c.contentID
LEFT JOIN Movies m ON m.contentID = c.contentID
LEFT JOIN Series s ON s.contentID = c.contentID
LEFT JOIN LiveStreams ls ON ls.contentID = c.contentID
LEFT JOIN ShortContents sc ON sc.contentID = c.contentID
GROUP BY
    c.contentID, c.title, c.genre, c.language, c.duration, c.date,
    m.contentID, s.contentID, ls.contentID;
```

Usage example:

-- Top 10 movies by user rating with at least 50 ratings

```
SELECT title, avgUserRating, totalUserRatings, avgCriticScore
FROM ContentStatistics
WHERE contentType = 'Movie' AND totalUserRatings >= 50
ORDER BY avgUserRating DESC
LIMIT 10;
```

View 2 – UserActivitySummary:

Purpose

Gives a comprehensive activity overview for each user: watched items, average rating, watchlists, followers, clubs, and badges. Eliminates the need for multiple joins in profile or analytics queries.

Use Case

Used in user profile pages, leaderboards, and admin dashboards.

View:

```
CREATE VIEW UserActivitySummary AS
SELECT
    u.userID,
    u.username,
    u.email,
    COUNT(DISTINCT wl2.contentID) AS watchedCount,
    ROUND(AVG(rl.rating), 2) AS avgRating,
    COUNT(DISTINCT wl.watchListID) AS watchListCount,
    COUNT(DISTINCT f.followedID) AS followingCount,
    COUNT(DISTINCT f2.followerID) AS followerCount,
    COUNT(DISTINCT uc.clubID) AS clubCount,
    COUNT(DISTINCT ub.badgeID) AS badgeCount,
    COUNT(DISTINCT or2.reviewID) AS officialReviewCount
FROM Users u
LEFT JOIN WatchLog wl2 ON wl2.userID = u.userID
LEFT JOIN RatingLog rl ON rl.userID = u.userID
LEFT JOIN WatchList wl ON wl.userID = u.userID
LEFT JOIN Follower f ON f.followerID = u.userID
LEFT JOIN Follower f2 ON f2.followedID = u.userID
LEFT JOIN UserClub uc ON uc.userID = u.userID
LEFT JOIN UserBadge ub ON ub.userID = u.userID
LEFT JOIN OfficialReviews or2 ON or2.criticUserID = u.userID
GROUP BY u.userID, u.username, u.email;
```

Usage example:

-- Most active users

```
SELECT username, watchedCount, avgRating, badgeCount
FROM UserActivitySummary
ORDER BY watchedCount DESC
LIMIT 20;
```

View 3 – ContentBrowseView:

Purpose

Simplifies browsing queries by precomputing content type (Movie, Series, etc.) and storing basic filtering attributes.

Use Case

Used in home page, browse page, and filtering/search operations.

View:

```
CREATE VIEW ContentBrowseView AS
SELECT
  c.contentID,
  c.title,
  c.genre,
  c.language,
  c.date,
  c.globalScore,
  CASE
    WHEN m.contentID IS NOT NULL THEN 'Movie'
    WHEN s.contentID IS NOT NULL THEN 'Series'
    WHEN ls.contentID IS NOT NULL THEN 'LiveStream'
    ELSE 'ShortContent'
  END AS contentType
FROM Content c
LEFT JOIN Movies m ON m.contentID = c.contentID
LEFT JOIN Series s ON s.contentID = c.contentID
LEFT JOIN LiveStreams ls ON ls.contentID = c.contentID;
```

Usage example:

-- Browse latest content filtered by genre

```
SELECT *
FROM ContentBrowseView
WHERE genre = 'Drama'
ORDER BY date DESC;
```

View 4 – UserWatchHistoryView

Purpose:

Combines watch logs, ratings, and comments into a single structure for each user-content interaction.

Use Case:

Used in “Watch History” and “Recently Watched” sections.

View:

```
CREATE VIEW UserWatchHistoryView AS
SELECT
```

```

    wl.userID,
    wl.contentID,
    c.title,
    c.genre,
    rl.rating,
    wl.percentage,
    wl.timestamp,
    cl.comment
FROM WatchLog wl
JOIN Content c ON c.contentID = wl.contentID
LEFT JOIN RatingLog rl
    ON rl.userID = wl.userID AND rl.contentID = wl.contentID
LEFT JOIN CommentLog cl
    ON cl.userID = wl.userID AND cl.contentID = wl.contentID;

```

Usage example:

-- Get user watch history

```

SELECT title, rating, percentage, timestamp
FROM UserWatchHistoryView
WHERE userID = :userID
ORDER BY timestamp DESC;

```

View 5 – WatchListSummary

Purpose:

Provides watchlist metadata along with the number of contents inside each list.

Use Case:

Used in “My WatchLists” page and analytics.

View:

```

CREATE VIEW WatchListSummary AS
SELECT
    wl.watchListID,
    wl.userID,
    wl.title,
    wl.visibility,
    COUNT(wlc.contentID) AS contentCount
FROM WatchList wl
LEFT JOIN WatchListContent wlc
    ON wlc.watchListID = wl.watchListID
GROUP BY wl.watchListID, wl.userID, wl.title, wl.visibility;

```

Usage example:

-- List user watchlists

```
SELECT title, visibility, contentCount
FROM WatchListSummary
WHERE userID = :userID;
```

View 6 – ClubMembersView

Purpose:

Provides club membership details including usernames and join dates.

Use Case:

Used in club management and member listing.

View:

```
CREATE VIEW ClubMembersView AS
SELECT
    uc.clubID,
    u.userID,
    u.username,
    uc.joinDate
FROM UserClub uc
JOIN Users u ON u.userID = uc.userID;
```

Usage example:

-- Get members of a club

```
SELECT username, joinDate
FROM ClubMembersView
WHERE clubID = :clubID;
```

View 7 – UpcomingWatchParties

Purpose:

Lists all upcoming watch parties with participant counts.

Use Case:

Used in event dashboards and club pages.

View:

```
CREATE VIEW UpcomingWatchParties AS
SELECT
    wp.partyID,
    wp.clubID,
    c.title,
    wp.date,
    wp.authorizedCinema,
```

```

    wp.capacity,
    COUNT(wpu.userID) AS joined
FROM WatchParty wp
JOIN Content c ON c.contentID = wp.contentID
LEFT JOIN WatchPartyUser wpu ON wpu.partyID = wp.partyID
WHERE wp.date > NOW()
GROUP BY
    wp.partyID, wp.clubID, c.title, wp.date, wp.authorizedCinema, wp.capacity;

```

Usage example:

-- List upcoming events for a club

```

SELECT *
FROM UpcomingWatchParties
WHERE clubID = :clubID
ORDER BY date;

```

View 8 – UserBadgesView

Purpose:

Provides badge information earned by users with descriptive metadata.

Use Case:

Used in user profiles features.

View:

```

CREATE VIEW UserBadgesView AS
SELECT
    ub.userID,
    b.id AS badgeID,
    b.name,
    b.explanation,
    ub.earnedAt
FROM UserBadge ub
JOIN Badge b ON b.id = ub.badgeID;

```

Usage example:

-- Get user badges

```

SELECT name, explanation, earnedAt
FROM UserBadgesView
WHERE userID = :userID;

```

2.4.2 Triggers

Trigger 1 - watchlist_content_added

Explanation:

This trigger is activated after a new row is inserted into WatchListContent. Its purpose is to automatically increase the numOfContent value of the related watchlist. In this way, the system keeps the stored number of items in each watchlist consistent with the actual contents of the junction table.

Use Case:

Used when a user adds a content item into a watchlist. The application only inserts the new relation into WatchListContent; the trigger automatically updates the total content count of that watchlist.

Trigger:

```
CREATE TRIGGER watchlist_content_added
AFTER INSERT ON WatchListContent
FOR EACH ROW
BEGIN
    UPDATE WatchList
    SET numOfContent = numOfContent + 1
    WHERE watchListID = NEW.watchListID;
END
```

Trigger 2 - watchlist_content_removed**Explanation:**

This trigger is activated after a row is deleted from WatchListContent. It decreases the numOfContent field of the corresponding watchlist. The CASE expression prevents the count from becoming negative, which adds an extra level of safety.

Use Case:

Used when a user removes a content item from a watchlist. The trigger ensures that the watchlist's stored content count is updated immediately and remains valid.

Trigger:

```
CREATE TRIGGER watchlist_content_removed
AFTER DELETE ON WatchListContent
FOR EACH ROW
BEGIN
    UPDATE WatchList
    SET numOfContent = CASE
        WHEN numOfContent > 0 THEN numOfContent - 1
        ELSE 0
    END
    WHERE watchListID = OLD.watchListID;
END
```

Trigger 3 - club_member_added

Explanation:

This trigger runs after a new membership record is inserted into UserClub. It increases the numOfMembers attribute of the related club so that the stored club size always reflects the actual number of users who joined.

Use Case:

Used when a user joins a club. Instead of manually updating the club table in application logic, the trigger automatically keeps the membership count correct.

Trigger:

```
CREATE TRIGGER club_member_added
AFTER INSERT ON UserClub
FOR EACH ROW
BEGIN
    UPDATE Club
    SET numOfMembers = numOfMembers + 1
    WHERE clubID = NEW.clubID;
END
```

Trigger 4 - club_member_removed

Explanation:

This trigger is executed after a membership row is deleted from UserClub. It decreases the numOfMembers value of the related club. Similar to the watchlist removal trigger, it also prevents the count from dropping below zero.

Use Case:

Used when a user leaves a club. The trigger guarantees that the stored member count of the club remains synchronised with the junction table.

Trigger:

```
CREATE TRIGGER club_member_removed
AFTER DELETE ON UserClub
FOR EACH ROW
BEGIN
    UPDATE Club
    SET numOfMembers = CASE
        WHEN numOfMembers > 0 THEN numOfMembers - 1
        ELSE 0
    END
    WHERE clubID = OLD.clubID;
```

END

Trigger 5 - critic_score_after_insert

Explanation:

This trigger is fired after a new review is inserted into OfficialReviews. It recalculates the average critic score of the related content and updates the criticScore field in the Content table. This ensures that every newly submitted critic review is immediately reflected in the content's aggregate score.

Use Case:

Used when a critic submits a new official review for a movie, series, or other content item. The system automatically refreshes the average critic score shown on the content page.

Trigger:

```
CREATE TRIGGER critic_score_after_insert
AFTER INSERT ON OfficialReviews
FOR EACH ROW
BEGIN
    UPDATE Content
    SET criticScore = (
        SELECT AVG(points)
        FROM OfficialReviews
        WHERE contentID = NEW.contentID
    )
    WHERE contentID = NEW.contentID;
END
```

Trigger 6 - critic_score_after_update

Explanation:

This trigger runs after an existing row in OfficialReviews is updated. It recomputes the average of critic points for the related content item and stores the new result in Content.criticScore. Thus, any modification to an official review is directly reflected in the overall critic evaluation.

Use Case:

Used when a critic edits the score of a previously submitted review. The trigger ensures that the displayed critic score always reflects the latest review values.

Trigger:

```
CREATE TRIGGER critic_score_after_update
AFTER UPDATE ON OfficialReviews
FOR EACH ROW
BEGIN
    UPDATE Content
```

```

SET criticScore = (
    SELECT AVG(points)
    FROM OfficialReviews
    WHERE contentID = NEW.contentID
)
WHERE contentID = NEW.contentID;
END

```

Trigger 7 - critic_score_after_delete

Explanation:

This trigger is activated after an official review is deleted from OfficialReviews. It recalculates the average critic score for the related content item based on the remaining reviews. Therefore, removing a review does not leave outdated aggregate values in the Content table.

Use Case:

Used when an official review is removed by the system or an administrator. The trigger automatically updates the critic score of the content item.

Trigger:

```

CREATE TRIGGER critic_score_after_delete
AFTER DELETE ON OfficialReviews
FOR EACH ROW
BEGIN
    UPDATE Content
    SET criticScore = (
        SELECT AVG(points)
        FROM OfficialReviews
        WHERE contentID = OLD.contentID
    )
    WHERE contentID = OLD.contentID;
END

```

Trigger 8 - global_score_after_insert

Explanation:

This trigger executes after a new user rating is inserted into RatingLog. It recalculates the overall user rating average of the related content item and updates the globalScore field in Content. This allows the system to keep a ready-to-display global score without recalculating it in every query.

Use Case:

Used when a user rates a content item. The trigger ensures that the new rating immediately affects the global score shown to all users.

Trigger:

```
CREATE TRIGGER global_score_after_insert
AFTER INSERT ON RatingLog
FOR EACH ROW
BEGIN
    UPDATE Content
    SET globalScore = (
        SELECT AVG(rating)
        FROM RatingLog
        WHERE contentID = NEW.contentID
    )
    WHERE contentID = NEW.contentID;
END
```

Trigger 9 - global_score_after_update**Explanation:**

This trigger runs after an existing rating in RatingLog is updated. It recalculates the average user rating for the corresponding content and updates Content.globalScore. This keeps aggregate rating data accurate even when users edit their ratings.

Use Case:

Used when a user changes their previous rating for a content item. The trigger ensures that the stored global score stays current.

Trigger:

```
CREATE TRIGGER global_score_after_update
AFTER UPDATE ON RatingLog
FOR EACH ROW
BEGIN
    UPDATE Content
    SET globalScore = (
        SELECT AVG(rating)
        FROM RatingLog
        WHERE contentID = NEW.contentID
    )
    WHERE contentID = NEW.contentID;
END
```

Trigger 10 - global_score_after_delete**Explanation:**

This trigger is activated after a rating row is deleted from RatingLog. It recomputes the average score of the related content item using the remaining ratings and updates Content.globalScore. This prevents stale aggregate values after a rating is removed.

Use Case:

Used when a user deletes their rating or when a rating is removed by the system. The trigger automatically updates the content's global score.

Trigger:

```
CREATE TRIGGER global_score_after_delete
AFTER DELETE ON RatingLog
FOR EACH ROW
BEGIN
    UPDATE Content
    SET globalScore = (
        SELECT AVG(rating)
        FROM RatingLog
        WHERE contentID = OLD.contentID
    )
    WHERE contentID = OLD.contentID;
END
```

2.4.3 Constraints

Explanation: A user must not be allowed to follow themselves. This constraint protects integrity: a self-follow would inflate follower counts, distort recommendation queries, and create logical errors in feed generation.

```
ALTER TABLE Follower
ADD CONSTRAINT chk_no_self_follow
CHECK (followerID <> followedID);
```

Explanation: A WatchParty must have a positive seating capacity. Allowing zero or negative capacity would make the party permanently full and prevent any user from joining, which is meaningless.

```
ALTER TABLE WatchParty
ADD CONSTRAINT chk_positive_capacity
CHECK (capacity > 0);
```

Explanation: Points must be between 1 and 10.

```
ALTER TABLE OfficialReviews
ADD CONSTRAINT chk_valid_points
CHECK (points >= 0 AND points <= 10);
```

Explanation: Watch percentage can not be higher than 100 percent and lower than 0 as expected.

```
ALTER TABLE WatchLog
```

```
ADD CONSTRAINT chk_valid_percentage
```

```
CHECK (percentage >= 0 AND percentage <= 100);
```

Explanation: End dates must be greater than start date as expected.

```
ALTER TABLE GroupChallenges
```

```
ADD CONSTRAINT chk_challenge_dates
```

```
CHECK (endDate > startDate);
```

2.5 Implementation Plan

This project is built on a three-tier web architecture containerized with **Docker**. **MySQL** handles all the data storage using a relational, table-based structure queried via SQL. **Flask (Python)** backend exposes RESTful API endpoints that process logic, communicate with the database, and return JSON responses. The **React.js** frontend consumes these APIs to render a dynamic, component-based user interface that updates in real time. All three services run in isolated Docker containers orchestrated by Docker Compose, ensuring a consistent and portable environment that can be launched with a single compose-up command.